

Chương 1. Luồng cực đại trên mạng

1.1. Các khái niệm và bài toán

1.1.1. Mạng

Mạng (flow network) là một bộ năm $G = (V, E, c, s, t)$, trong đó:

- V và E lần lượt là tập đỉnh và tập cung của một đồ thị có hướng, $|V| = n$ và $|E| = m$ lần lượt là số đỉnh và số cung của mạng, các đỉnh đánh số từ 0 tới $n - 1$.
- s và t là hai đỉnh phân biệt thuộc V , s gọi là *đỉnh phát* (source) và t gọi là *đỉnh thu* (sink).
- c là một hàm số xác định trên tập cung E

$$\begin{aligned} c: E &\rightarrow \mathbb{R}_{\geq 0} \\ e &\mapsto c(e) \end{aligned}$$

gán cho mỗi cung $e \in E$ một số không âm gọi là *sức chứa* (capacity) $c(e) \geq 0$.

Để thuận tiện cho việc trình bày, với $A \subseteq E$ là một tập các cung, $X, Y \subseteq V$ là hai tập đỉnh, $f: E \rightarrow \mathbb{R}$ là một hàm xác định trên tập cung E , ta quy ước các ký hiệu sau:

- $f(A)$ là tổng các giá trị hàm f trên các cung của A :

$$f(A) = \sum_{e \in A} f(e)$$

- $X \rightarrow Y$ là tập các cung nối X sang Y

$$X \rightarrow Y = \{e = (u, v) \in E : u \in X, v \in Y\}$$

Trong trường hợp X chỉ gồm một đỉnh x , ta dùng ký hiệu $x \rightarrow Y$ thay cho $\{x\} \rightarrow Y$, tương tự như vậy trong trường hợp Y chỉ gồm một đỉnh y , ta dùng ký hiệu $X \rightarrow y$ thay cho $X \rightarrow \{y\}$.

- Theo đó, $f(X \rightarrow Y)$ là tổng các giá trị hàm f trên tất cả các cung nối từ X sang Y :

$$f(X \rightarrow Y) = \sum_{e \in X \rightarrow Y} f(e)$$

Bổ đề 1-1

Với ba tập đỉnh $X, Y, Z \subseteq V, X \cap Y = \emptyset, f: E \rightarrow \mathbb{R}$ là hàm xác định trên tập cung e . Khi đó:

$$\begin{aligned} f(X \cup Y \rightarrow Z) &= f(X \rightarrow Z) + f(Y \rightarrow Z) \\ f(Z \rightarrow X \cup Y) &= f(Z \rightarrow X) + f(Z \rightarrow Y) \end{aligned}$$

Chứng minh

Phép chứng minh đơn giản dùng định nghĩa:

$$\begin{aligned} f(X \cup Y \rightarrow Z) &= \sum_{e \in X \cup Y \rightarrow Z} f(e) \\ &= \sum_{e \in X \rightarrow Z} f(e) + \sum_{e \in Y \rightarrow Z} f(e) \\ &= f(X \rightarrow Z) + f(Y \rightarrow Z) \\ f(Z \rightarrow X \cup Y) &= \sum_{e \in Z \rightarrow X \cup Y} f(e) \\ &= \sum_{e \in Z \rightarrow X} f(e) + \sum_{e \in Z \rightarrow Y} f(e) \\ &= f(Z \rightarrow X) + f(Z \rightarrow Y) \quad \blacksquare \end{aligned}$$

1.1.2. Luồng

Luồng (flow) trên mạng G là một hàm:

$$\begin{aligned} f: E &\rightarrow \mathbb{R}_{\geq 0} \\ e &\mapsto \varphi(e) \end{aligned}$$

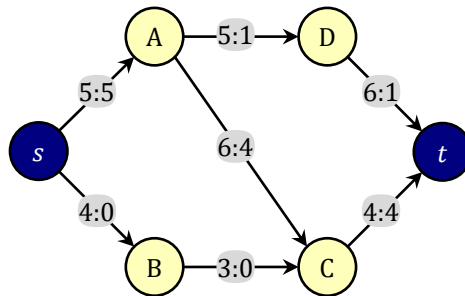
gán cho mỗi cung e một số thực không âm $f(e)$ gọi là luồng trên cung e thỏa mãn hai ràng buộc sau đây:

- ☀ Ràng buộc về sức chứa (*Capacity constraint*): Luồng trên mỗi cung không được vượt quá sức chứa của cung đó: $\forall e \in E: 0 \leq f(e) \leq c(e)$
- ☀ Ràng buộc về tính bảo toàn (*Flow conservation*): Với mọi đỉnh u không phải đỉnh phát và cũng không phải đỉnh thu, tổng luồng trên các cung đi vào u bằng tổng luồng trên các cung đi ra khỏi u : $\forall v \in V \setminus \{s, t\}: f(V \rightarrow u) = f(u \rightarrow V)$

Giá trị của một luồng định nghĩa bằng tổng luồng trên các cung đi ra khỏi đỉnh phát trừ đi tổng luồng trên các cung đi vào đỉnh thu:

$$|f| = f(s \rightarrow V) - f(V \rightarrow t)$$

Chú ý rằng luồng trên mỗi cung luôn là số không âm nhưng giá trị luồng hoàn toàn có thể là số âm. Một ví dụ đơn giản là khi ta đảo vai trò đỉnh phát và đỉnh thu trên mạng thì giá trị luồng bị đổi dấu. Điều này sẽ được chỉ ra trong các định lý tiếp theo.



Hình 1-1. Ví dụ về một mạng và luồng giá trị 5, nhãn ghi trên cung là sức chứa:luồng

Một số tài liệu khác đưa vào thêm ràng buộc: đỉnh phát s không có cung đi vào và đỉnh thu t không có cung đi ra. Khi đó giá trị luồng bằng tổng luồng trên các cung đi ra khỏi đỉnh phát. Cách hiểu này có thể quy về một trường hợp riêng của định nghĩa.

Bài toán luồng cực đại trên mạng (maximum-flow problem): Cho một mạng G với đỉnh phát s và đỉnh thu t , hàm sức chứa c , hãy tìm một luồng có giá trị lớn nhất trên mạng G .

Mô hình trực quan nhất của luồng là các đường ống dẫn nước từ trạm nguồn s tới nơi trạm đích t . Các đỉnh khác của mạng là các trạm trung chuyển và các cung là đường ống dẫn nước một chiều từ một trạm tới một trạm khác. Sức chứa của đường ống là tiết diện của ống, tỉ lệ thuận với lưu lượng nước tối đa có thể chảy qua trong một đơn vị thời gian. Trạm trung gian không có khả năng chứa nước, do vậy lượng nước chảy vào trạm trung gian phải được phát tán toàn bộ và ngay lập tức sang trạm khác qua các đường ống. Ta có ngay các ràng buộc: Trong mỗi đơn vị thời gian, lượng nước lưu thông qua mỗi đường ống không được vượt quá sức chứa của đường ống và bao nhiêu nước đi vào một trạm trung chuyển thì cũng phải có bấy nhiêu nước đi ra khỏi trạm trung chuyển đó. Vấn đề đặt ra là điều khiển lượng nước vào các đường ống một cách hợp lý để tổng lượng nước chuyển đi

trong mỗi đơn vị thời gian từ s tới t là lớn nhất. Các vấn đề như phân luồng giao thông để tránh tắc nghẽn, lắp đặt mạch điện để đảm bảo không gặp phải sự cố quá tải đường dây đều là những ví dụ thực tế của bài toán luồng cực đại trên mạng.

1.1.3. Cung đối

Với mỗi cung $e = (u, v) \in E$, ta thêm vào một cung giả, đi ngược chiều, ký hiệu $-e = (v, u)$, gọi là cung đối của cung e . Sức chứa của cung giả này được đặt bằng 0. Ta cũng coi cung e là cung đối của cung $-e$: $e = -(-e)$. Số cung của mạng được nhân đôi.

Nếu trên mạng đã có sẵn một luồng, ta chỉ cần đặt luồng trên các cung giả bằng 0 thì vẫn duy trì được tất cả các điều kiện của luồng*. Từ giờ trở đi, các thuật toán sẽ làm việc trên một mạng gồm các cặp cung đối như vậy. Để phân biệt với cung giả, các cung trên mạng ban đầu sẽ được gọi là cung thật. Trên các hình vẽ biểu thị luồng, ta sẽ chỉ vẽ các cung thật, các cung giả chỉ được vẽ trong một số trường hợp đặc biệt

1.1.4. Mạng dư lượng

Với f là một luồng trên mạng $G = (V, E, c, s, t)$. Ta xét mạng G_f cũng là mạng G nhưng với hàm sức chứa mới cho bởi:

$$\begin{aligned} c_f: E_f &\rightarrow \mathbb{R}_{\geq 0} \\ e &\mapsto c_f(e) = \begin{cases} c(e) - f(e), & \text{nếu } e \text{ là cung thật} \\ f(-e) & , \text{nếu } e \text{ là cung giả} \end{cases} \\ &= c(e) - f(e) + f(-e) \end{aligned}$$

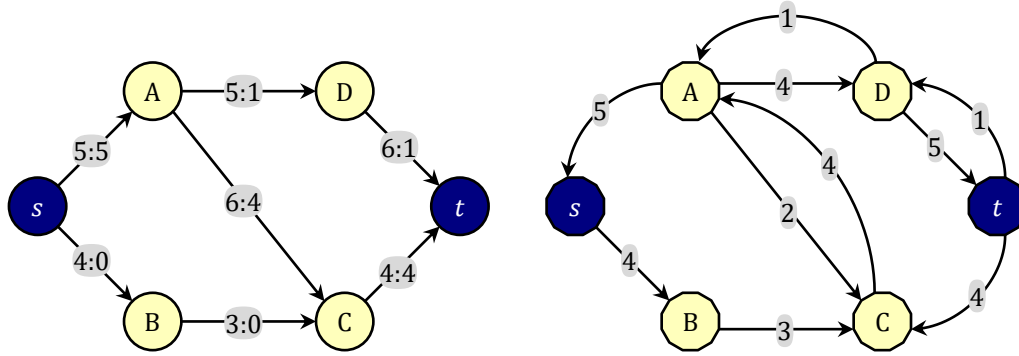
Sức chứa của một cung trên mạng dư lượng được gọi là *dư lượng (residual capacity)* của cung đó. Mỗi cung thật e với sức chứa $c(e)$ và luồng $f(e)$ sẽ cho biết dư lượng của 2 cung: cung e với dư lượng $c(e) - f(e)$ và cung đối $-e$ với dư lượng $f(e)$. Dư lượng $c_f(e)$ là lượng luồng tối đa có thể thêm vào luồng trên cung e mà không làm nó vượt quá sức chứa, còn dư lượng $c_f(-e)$ là lượng luồng tối đa có thể bớt đi trên cung e mà không làm nó bị âm. Bởi sức chứa và luồng trên các cung giả đều bằng 0, biểu thức dư lượng có thể viết gọn lại thành:

$$c_f(e) = c(e) - f(e) + f(-e)$$

Trên mạng dư lượng, một cung gọi là *cung bão hòa (saturated edge)* nếu dư lượng của cung đó bằng 0, ngược lại cung đó gọi là *cung dư (residual edge)*. Ký hiệu E_f là tập các cung dư trên mạng dư lượng G_f . Một đường dư (*residual path*) trên G_f là một đường đi chỉ chứa các cung dư.

Các cung bão hòa của mạng G có dư lượng 0, cung này ít có ý nghĩa trong thuật toán nên chúng ta sẽ chỉ vẽ các cung dư ($\in E_f$) trong các hình vẽ.

* Có một cách làm khác là đặt luồng trên cung giả bằng số đối của luồng trên cung thật: $f(-e) = -f(e)$. Cách làm này cho các chứng minh đơn giản hơn, nhưng lại mất đi tính tự nhiên của luồng, cần sửa đổi khái niệm ràng buộc luồng.



Hình 1-2. Một luồng trên mạng và các cung dư trên mạng dư lượng tương ứng

Hình 1-2 là một ví dụ mạng, luồng và mạng dư lượng tương ứng. Trên sơ đồ mạng và luồng, ta không vẽ các cung giả, còn trên sơ đồ mạng dư lượng, ta không vẽ các cung bão hòa. Mạng có cung (A, C) với sức chứa 6 và luồng 5 thì trên mạng dư lượng, ta có cung thật (A, C) với dư lượng $6 - 5 = 1$ và cung giả (C, A) với sức chứa 5.

Mạng dư lượng là thành phần rất quan trọng trong các thuật toán tìm luồng cực đại. Tuy vậy, việc duy trì toàn bộ sức chứa, luồng, dư lượng khá bất tiện khi cập nhật. Vì lý do đó, trong các chương trình cài đặt, ta **chỉ duy trì thông tin về mạng dư lượng**. Sức chứa và luồng khi cần thiết sẽ được suy ra từ dư lượng theo cách sau:

- Sức chứa của một cung thật e trên mạng ban đầu bằng tổng dư lượng của nó và dư lượng của cung giả tương ứng:

$$c_f(e) + c_f(-e) = c(e) - f(e) + f(e) = \boxed{c(e)}$$

- Luồng của một cung thật e trên mạng ban đầu bằng sức chứa trên cung đối:

$$c_f(-e) = \boxed{f(e)}$$

Tất nhiên sức chứa và luồng trên các cung giả đều bằng 0, những thông tin này không có ý nghĩa. Cung giả chỉ có ý nghĩa về dư lượng mà thôi.

1.1.5. Phép tăng luồng trên cung

Giả sử f là một luồng trên mạng $G = (V, E, c, s, t)$. Tưởng tượng rằng có 1 đơn vị luồng di chuyển theo một đường đi nào đó. Đường đi này có thể chứa cả cung thật và cung giả, tức là đơn vị luồng này sẽ di chuyển xuôi hoặc ngược chiều các cung thật trên mạng ban đầu: nếu di chuyển xuôi chiều cung thật sẽ làm luồng trên cung đó tăng lên 1, còn nếu di chuyển ngược chiều cung thật sẽ làm luồng trên cung đó giảm đi 1.

Từ bây giờ, trong các thuật toán, hành động “**Đẩy thêm Δ đơn vị luồng theo cung e** ” hay ngắn gọn: “**Thêm Δ luồng theo cung e** ” có nghĩa là tăng $f(e)$ lên Δ nếu e là cung thật hoặc giảm $f(-e)$ đi Δ nếu e là cung giả, hành động này mô tả bằng hàm $\text{IncFlow}(e, \Delta)$ như sau:

```

1 void IncFlow(e ∈ E, Δ)
2 {
3     if (e là cung thật)
4         f(e) += Δ;
5     else
6         f(-e) -= Δ;
7 }
```

Đối chiếu trên mạng dư lượng, từ biểu thức của dư lượng trên hai cung đối nhau:

$$c_f(e) = c(e) - f(e) + f(-e)$$

$$c_f(-e) = c(-e) - f(-e) + f(e)$$

Ta thấy rằng bất kể e là cung thật hay cung giả, dù hàm $\text{IncFlow}(e, \Delta)$ tăng $f(e)$ thêm Δ hay giảm $f(-e)$ đi Δ , ta đều có $c_f(e)$ giảm đi Δ và $c_f(-e)$ tăng lên Δ . Trong các chương trình cài đặt, chỉ có thông tin về dư lượng được duy trì, khi đó hàm $\text{IncFlow}(e, \Delta)$ có thể viết ngắn gọn như sau:

```

1 void IncFlow(e ∈ E, Δ)
2 {
3     c_f(e) -= Δ;
4     c_f(-e) += Δ;
5 }
```

Hành động “**Đẩy thêm Δ đơn vị luồng theo đường đi P** ” được hiểu là đẩy Δ đơn vị luồng theo các cung dọc đường đi, ta sẽ hiểu rõ hơn về hành động này ở mục nói về đường tăng luồng và thuật toán Ford-Fulkerson.

1.2. Thuật toán Ford-Fulkerson

1.2.1. Đường tăng luồng

Với f là một luồng trên mạng $G = (V, E, c, s, t)$. Gọi P là một đường đi đơn từ s tới t trên mạng dư lượng G_f . *Dư lượng (residual capacity)* của đường P được định nghĩa bằng dư lượng nhỏ nhất của các cung dọc trên đường P :

$$\min_{e \in P} \{c_f(e)\}$$

Vì dư lượng trên các cung đều là số không âm nên dư lượng của đường P luôn là số không âm. Nếu đường đi P sẽ chỉ gồm các cung dư, dư lượng của đường P là một số dương, khi ấy P được gọi là một *đường tăng luồng (augmenting path)*.

1.2.2. Thuật toán Ford-Fulkerson

Thuật toán Ford-Fulkerson [1] để tìm luồng cực đại trên mạng dựa trên cơ chế tăng luồng dọc theo đường tăng luồng:

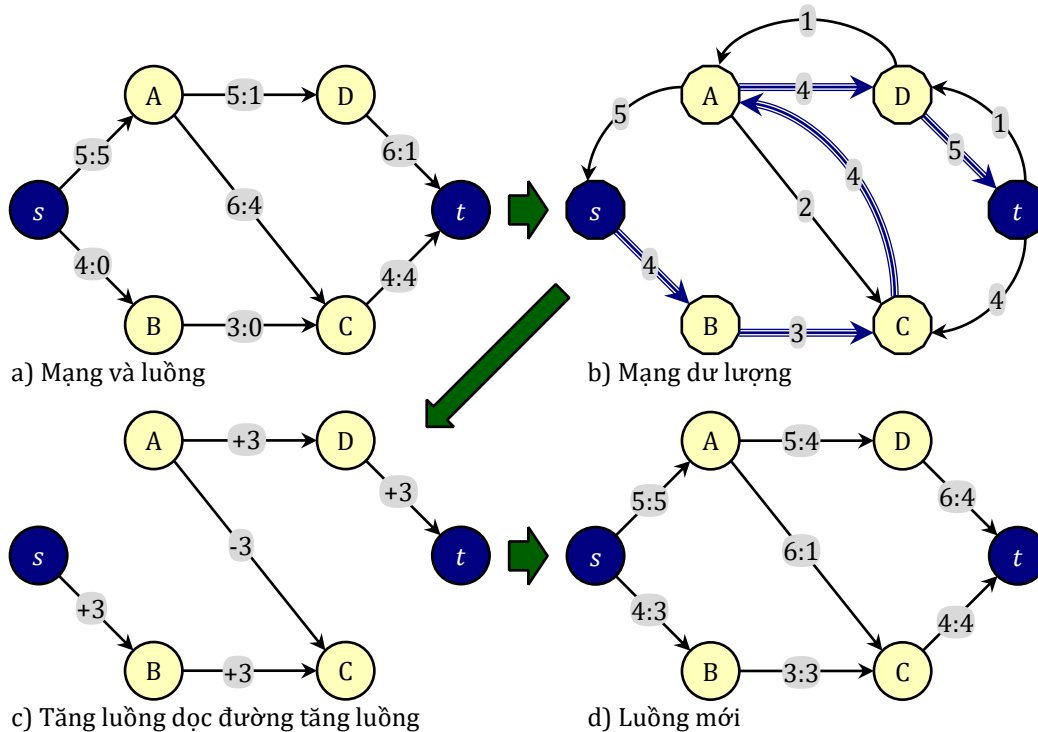
- ☀ Khởi tạo: thuật toán bắt đầu một luồng bất kỳ trên mạng (chẳng hạn luồng trên mọi cung đều bằng 0)
- ☀ Lặp: Tìm đường tăng luồng P trên mạng dư lượng. Nếu tìm thấy, tính Δ là dư lượng của đường P , sau đó đẩy một lượng luồng Δ theo đường P .
- ☀ Kết thúc: Vòng lặp kết thúc khi mạng dư lượng không còn đường tăng luồng, luồng hiện tại trên mạng là luồng cực đại

```

1 «Khởi tạo một luồng f bất kỳ»;
2 while («Tìm được đường tăng luồng P: s→t trên mạng dư lượng»)
3 {
4     Δ = «Dư lượng của đường P»;
5     for (∀ e ∈ P)
6         IncFlow(e, Δ);
7 }
8 Output ← f;
```

Hình 1-3 là ví dụ về cơ chế tăng luồng trên mạng:

- ☀ Hình a) là mạng với đỉnh phát s , đỉnh thu t và luồng giá trị 5 (ta không vẽ các cung giả)
- ☀ Hình b) là mạng dư lượng G_f (ta chỉ vẽ các cung dư, các cung giả được vẽ bằng nét cong), đường đi $P = \langle s, B, C, A, D, t \rangle$ được chọn làm đường tăng luồng, dư lượng của P bằng 3 (dư lượng của cung (B, C)).
- ☀ Hình c) là cơ chế tăng luồng: Luồng trên các cung thật dọc đường P được tăng lên 3 và luồng trên các cung đối của các cung giả dọc đường P bị giảm đi 3.
- ☀ Hình d) là luồng mới thu được với giá trị 8.



Hình 1-3. Cơ chế tăng luồng của thuật toán Ford-Fulkerson

1.2.3. Cài đặt

Input

Mạng với n đỉnh, m cung thật. Các đỉnh đánh số từ 0 tới $n - 1$, các cung đánh số từ 0 tới $m - 1$

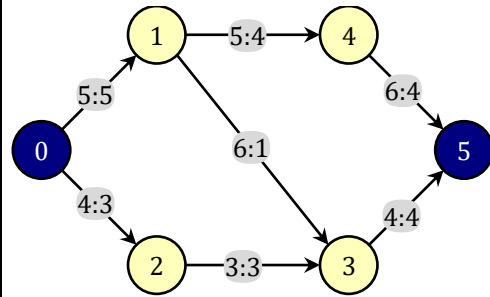
- ☀ Dòng 1: Số đỉnh $n \leq 10^4$, số cung $m \leq 10^5$, đỉnh phát s , đỉnh thu t .
- ☀ m dòng tiếp theo: mỗi dòng chứa ba số nguyên dương u, v, c tương ứng với một cung nối từ u tới v với sức chứa $c \leq 10^9$.

Input chỉ cho sức chứa các cung thật, các cung giả phải được chương trình tự thêm vào với sức chứa 0 trước khi thực hiện thuật toán.

Output

Luồng cực đại trên mạng

Sample Input	Sample Output
6 7 0 5	Value of flow: 8
0 1 5	$e[0] = (0, 1): c = 5, f = 5$
0 2 4	$e[1] = (0, 2): c = 4, f = 3$
1 3 6	$e[2] = (1, 3): c = 6, f = 1$
1 4 5	$e[3] = (1, 4): c = 5, f = 4$
2 3 3	$e[4] = (2, 3): c = 3, f = 3$
3 5 4	$e[5] = (3, 5): c = 4, f = 4$
4 5 6	$e[6] = (4, 5): c = 6, f = 4$



• Tổ chức dữ liệu

Để duy trì thông tin về các cặp cung đối nhau trên mạng dư lượng, tất cả m cung thật và cung giả đối của chúng được chứa trong danh sách $e[0 \dots 2m - 1]$. Các cung thật được lưu trữ ở các vị trí chẵn, cung giả của chúng được lưu ở vị trí lẻ liền sau. Tức là các cặp cung đối được bố trí theo thứ tự:

$$\begin{aligned}
 e[0] &= -e[1] \\
 e[2] &= -e[3] \\
 &\dots \\
 e[2m-2] &= -e[2m-1]
 \end{aligned}$$

Khi đó mỗi cung $e[i]$ sẽ là cung đối với cung $e[i^1]$ trong đó toán tử 1 là phép toán XOR (eXclusive OR). Phép tính i^1 đảo đi bit đơn vị của i , cho kết quả là số lẻ liền sau i nếu i chẵn hoặc số chẵn liền trước i nếu i lẻ.

Mỗi cung là một cấu trúc chứa ba trường x, y, cf trong đó x, y là đỉnh đầu và đỉnh cuối của cung, cf là dư lượng của cung. Ban đầu với luồng 0, dư lượng trên các cung thật (chỉ số chẵn) khởi tạo bằng chính sức chứa trên cung đó và dư lượng trên các cung giả (chỉ số lẻ) được khởi tạo bằng 0. Như đã trình bày trong mục 1.1.4, ta không lưu trữ luồng và sức chứa trên cung mà chỉ tính toán những thông tin này khi in kết quả.

Chỉ số của các cung đi ra khỏi u được lưu trong danh sách liên thuộc $a[u]$.

• Lưu vết và dò đường tăng luồng

Tại mỗi bước, thuật toán BFS được dùng để tìm đường tăng luồng, mỗi đỉnh v trên đường đi được lưu vết $trace[v]$ là chỉ số cung đi vào v trên đường tăng luồng. Dựa vào vết này, ta sẽ dò lại các cung để tăng luồng dọc theo đường tăng luồng tìm được.

Thuật toán BFS phải thực hiện nhiều lần, mỗi lần ta phải đặt lại tất cả các đỉnh vào trạng thái chưa thăm. Có một mẹo nhỏ để thực hiện điều này trong thời gian $O(1)$:

Trạng thái của mỗi đỉnh u là một số nguyên $state[u]$, ngoài ra có một biến nguyên $visited$. Ban đầu các biến $state[...]$ và $visited$ được khởi tạo bằng 0. Đỉnh u ở trạng thái đã thăm nếu $state[u] = visited$, ở trạng thái chưa thăm nếu $state[u] \neq visited$. Để khởi tạo các đỉnh ở trạng thái chưa thăm, ta chỉ cần tăng $visited$ lên 1 và trong thuật toán BFS, mỗi khi đánh dấu đỉnh u đã thăm ta chỉ cần gán $state[u] = visited$. Kỹ thuật này giúp cho mỗi lượt BFS tìm đường tăng luồng sẽ chỉ mất thời gian $O(m)$ thay vì $\Omega(n + m)$ trong trường hợp xấu nhất.

Edmonds và Karp [2] đã đề xuất mô hình cài đặt thuật toán Ford-Fulkerson trong đó đường tăng luồng được tìm bằng BFS. Vì thế người ta còn gọi thuật toán này là thuật toán Edmonds-Karp.

EDMONDS_KARP.cpp Thuật toán Edmonds-Karp

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <algorithm>
5  using namespace std;
6  const int MAX_N = 1e4;
7  const int MAX_M = 1e5;
8  const int MAX_C = 1e9;
9
10 struct TEdge //Cấu trúc một cung
11 {
12     int x, y; //Hai đỉnh đầu nút
13     int cf;    //dư lượng
14 };
15
16 int n, m, s, t;
17 TEdge e[MAX_M * 2];
18 vector<int> a[MAX_N]; //Danh sách liên thuộc
19 int trace[MAX_N];
20 int state[MAX_N], visited;
21
22 void Enter()
23 {
24     cin >> n >> m >> s >> t;
25     for (int i = 0; i < m * 2; i += 2)
26     {
27         int u, v, cap;
28         cin >> u >> v >> cap; //Đọc một cung (u, v) và sức chứa cap
29         e[i] = {u, v, cap}; //Thêm cung (u, v) với dư lượng cap
30         e[i + 1] = {v, u, 0}; //Thêm cung đối (v, u) với dư lượng 0
31         a[u].push_back(i);
32         a[v].push_back(i + 1);
33     }
34     fill(state, state + n, 0);
35     visited = 0;
36 }
37
38 bool FindPath() //Tìm đường tăng luồng bằng BFS
39 {
40     state[s] = ++visited; //s đã thăm, các đỉnh khác đều chưa thăm
41     queue<int> q({s}); //Hàng đợi ban đầu chỉ có đỉnh s
42     while (!q.empty())
43     {
44         int u = q.front(); q.pop(); //Lấy u khỏi hàng đợi
45         for (int i: a[u]) //Quét các cung e[i] ra khỏi u
46         {
47             int v = e[i].y; //e[i] = (u, v)
48             if (state[v] == visited || e[i].cf == 0) //v đã thăm hoặc e[i] bão hòa
49                 continue; //bỏ qua
50             state[v] = visited; //Đánh dấu v đã thăm
51             trace[v] = i; //Lưu vết: cung dư đi tới v là e[i]
52             if (v == t) //tìm ra đường tăng luồng
53                 return true;
54             q.push(v); //Đẩy v vào hàng đợi chờ xử lý
55         }
56     }
57     return false; //Không còn đường tăng luồng
58 }
```



```

59
60 void IncFlow(int i, int delta) //Đẩy thêm delta đơn vị luồng theo cung e[i]
61 {
62     e[i].cf -= delta; //Dư lượng cung e[i] giảm đi delta
63     e[i ^ 1].cf += delta; //Dư lượng cung đối tăng lên delta
64 }
65
66 void AugmentFlow() //Tăng luồng dọc đường tăng luồng
67 {
68     int delta = MAX_C;
69     //Tìm delta = dư lượng nhỏ nhất của các cung trên đường tăng luồng
70     for (int u = t; u != s; u = e[trace[u]].x)
71         delta = min(delta, e[trace[u]].cf);
72     //Tăng luồng dọc đường tăng luồng
73     for (int u = t; u != s; u = e[trace[u]].x)
74         IncFlow(trace[u], delta);
75 }
76
77 void Print() //In kết quả
78 {
79     long long flowVal = 0;
80     for (int i: a[s]) //Quét các cung ra khỏi s
81         if (i % 2 == 0) //Gặp cung thật ra khỏi s
82             flowVal += e[i ^ 1].cf; //Cộng luồng ra khỏi s
83         else //Gặp cung giả ra khỏi s, cung đối là cung thật đi vào s
84             flowVal -= e[i].cf; //Trừ luồng đi vào s
85     cout << "Value of flow: " << flowVal << '\n';
86     //Chỉ ra sức chứa và luồng trên các cung đã cho bằng thông tin về dư lượng
87     for (int i = 0; i < m * 2; i += 2) //Cung thật mang chỉ số chẵn trong e
88         cout << "e[" << i / 2 << "] = "
89             << "(" << e[i].x << ", " << e[i].y << "): "
90             << "c = " << e[i].cf + e[i ^ 1].cf << ", " //c(e) = cf(e) + cf(-e)
91             << "f = " << e[i ^ 1].cf << '\n'; //f(e) = cf(-e)
92 }
93
94 int main()
95 {
96     Enter(); //Nhập dữ liệu
97     while (FindPath()) //Thuật toán Ford-Fulkerson
98         AugmentFlow();
99     Print(); //In kết quả
100 }

```

Trong chương trình trên, giá trị luồng FlowVal được tính bằng định nghĩa: Tổng luồng trên các cung ra khỏi s trừ tổng luồng trên các cung đi vào s . Thực ra cách tính này hơi thừa: nếu ta khởi tạo luồng 0 thì thuật toán Ford-Fulkerson luôn cho luồng trên các cung đi vào s bằng 0. Thật vậy, các cung thật đi vào s không bao giờ nằm trên đường tăng luồng vì đường tăng luồng là đường đi đơn $s \rightsquigarrow t$, luồng trên cung đó không bao giờ tăng lên làm cho cung đối cũng luôn bão hòa và không bao giờ nằm trên đường tăng luồng. Cả cung thật và cung giả tương ứng với nó không nằm trên đường tăng luồng thì luồng trên cung thật bất biến (bằng 0).

Tuy nhiên ta vẫn tính giá trị luồng theo đúng định nghĩa để không mất công sửa đổi trong trường hợp luồng khởi tạo khác 0, hoặc dùng một thuật toán khác để tìm luồng cực đại.

1.2.4. Một số tính chất của mạng và luồng

Thuật toán Ford-Fulkerson không chỉ đưa ra một phương pháp tìm luồng cực đại, nó còn cung cấp rất nhiều cơ sở lý thuyết quan trọng về các tính chất của mạng và luồng. Ta tạm dừng nói về thuật toán để khảo sát những tính chất ấy.

Bổ đề 1-2

Cho f là một luồng trên mạng $G = (V, E, c, s, t)$, khi đó: $\forall X \subseteq V \setminus \{s, t\}$, ta có:

$$f(X \rightarrow V) = f(V \rightarrow X)$$

Chứng minh

$$\begin{aligned} f(X \rightarrow V) &= \sum_{u \in X} f(u \rightarrow V) \\ &= \sum_{u \in X} f(V \rightarrow u) \text{ (Do tính bảo toàn luồng qua } u \notin \{s, t\}) \\ &= f(V \rightarrow X) \blacksquare \end{aligned}$$

Định lý 1-3

Giá trị luồng trên mạng bằng tổng luồng trên các cung đi vào đỉnh thu trừ đi tổng luồng trên các cung ra khỏi đỉnh thu

Chứng minh

Giả sử f là một luồng trên mạng $G = (V, E, c, s, t)$, ta có:

$$\begin{aligned} f(\{s, t\} \rightarrow V) &= f(V \rightarrow V) - f(V \setminus \{s, t\} \rightarrow V) \text{ (Bổ đề 1-1)} \\ &= f(V \rightarrow V) - f(V \rightarrow V \setminus \{s, t\}) \text{ (Bổ đề 1-2)} \\ &= f(V \rightarrow \{s, t\}) \text{ (Bổ đề 1-1)} \end{aligned}$$

Vậy:

$$\begin{aligned} f(\{s, t\} \rightarrow V) &= f(V \rightarrow \{s, t\}) \\ \Rightarrow f(s \rightarrow V) + f(t \rightarrow V) &= f(V \rightarrow s) + f(V \rightarrow t) \\ \Rightarrow f(s \rightarrow V) - f(V \rightarrow s) &= f(V \rightarrow t) - f(t \rightarrow V) \\ \Rightarrow |f| &= f(V \rightarrow t) - f(t \rightarrow V) \blacksquare \end{aligned}$$

Định lý 1-4 (Tổng luồng)

Cho f là một luồng trên mạng $G = (V, E, c, s, t)$ và φ là một luồng trên mạng dư lượng tương ứng thì hàm:

$$\begin{aligned} f \uparrow \varphi: E &\rightarrow \mathbb{R}_{\geq 0} \\ e &\mapsto (f \uparrow \varphi)(e) = f(e) + \begin{cases} \varphi(e) - \varphi(-e), & \text{nếu } e \text{ là cung thật} \\ 0 & , \text{nếu } e \text{ là cung giả} \end{cases} \end{aligned}$$

là một luồng trên mạng G với giá trị luồng $|f \uparrow \varphi| = |f| + |\varphi|$.

Chứng minh

Về bản chất có thể coi luồng f trên mạng G là luồng chính và luồng φ trên mạng dư lượng G_f là luồng phụ mà ta muốn gộp vào luồng chính. Ký hiệu $f' = f \uparrow \varphi$. Ta chứng minh f' thỏa mãn các ràng buộc của luồng.

Ràng buộc về sức chứa: $\forall e \in E$,

Nếu e là cung giả thì hiển nhiên $f'(e) = 0 = c(e)$.

Nếu e là cung thật, cũng từ ràng buộc về sức chứa của luồng φ :

$$\begin{aligned} f'(e) &= f(e) + \varphi(e) - \varphi(-e) \\ &\geq f(e) - \varphi(-e) \\ &\geq f(e) - c_f(-e) \\ &= f(e) - f(e) \\ &= 0 \end{aligned}$$

$$\begin{aligned}
f'(e) &= f(e) + \varphi(e) - \varphi(-e) \\
&\leq f(e) + \varphi(e) \\
&\leq f(e) + c_f(e) \\
&= f(e) + c(e) - f(e) \\
&= c(e)
\end{aligned}$$

Ràng buộc về tính bảo toàn: từ biểu thức của luồng f' :

$$f'(e) = f(e) + \begin{cases} \varphi(e) - \varphi(-e), & \text{nếu } e \text{ là cung thật} \\ 0, & \text{nếu } e \text{ là cung giả} \end{cases}$$

Suy ra với $\forall e \in E$, bất kể e là cung thật hay cung giả, ta có:

$$f'(e) - f'(-e) = (f(e) - f(-e)) + (\varphi(e) - \varphi(-e))$$

Với $\forall u \in V$, từ quan sát cung $e \in u \rightarrow V$ thì cung $-e \in V \rightarrow u$ và ngược lại, ta có:

$$\begin{aligned}
f'(u \rightarrow V) - f'(V \rightarrow u) &= \sum_{e \in u \rightarrow V} (f'(e) - f'(-e)) \\
&= \sum_{e \in u \rightarrow V} (f(e) - f(-e)) + \sum_{e \in u \rightarrow V} (\varphi(e) - \varphi(-e)) \\
&= (f(u \rightarrow V) - f(V \rightarrow u)) + (\varphi(u \rightarrow V) - \varphi(V \rightarrow u))
\end{aligned}$$

Nếu $u \notin \{s, t\}$, từ tính bảo toàn của luồng f và luồng φ :

$$\begin{cases} f(u \rightarrow V) - f(V \rightarrow u) = 0 \\ \varphi(u \rightarrow V) - \varphi(V \rightarrow u) = 0 \end{cases}$$

Ta suy ra $f'(u \rightarrow V) - f'(V \rightarrow u) = 0$, hay:

$$f'(u \rightarrow V) = f'(V \rightarrow u)$$

Về giá trị luồng, thay $u = s$ ta có:

$$\begin{aligned}
|f'| &= f'(s \rightarrow V) - f'(V \rightarrow s) \\
&= \underbrace{(f(s \rightarrow V) - f(V \rightarrow s))}_{|f|} + \underbrace{(\varphi(s \rightarrow V) - \varphi(V \rightarrow s))}_{|\varphi|} \quad \blacksquare \\
&= |f| + |\varphi|
\end{aligned}$$

Định lý 1-5 (Luồng đường)

Cho f là một luồng trên mạng $G = (V, E, c, s, t)$, P là một đường tăng luồng trên G_f . Gọi Δ là dư lượng của đường P . Khi đó hàm:

$$\begin{aligned}
f_P: E &\rightarrow \mathbb{R}_{\geq 0} \\
e &\mapsto f_P(e) = \begin{cases} \Delta, & \text{nếu } e \in P \\ 0, & \text{nếu } e \notin P \end{cases}
\end{aligned}$$

là một luồng trên mạng dư lượng G_f với giá trị luồng $|f_P| = \Delta$.

Chứng minh

Nhắc lại rằng đường tăng luồng P là một đường đi đơn chỉ chứa các cung dư. Δ là dư lượng nhỏ nhất của một cung trên đường P (chắc chắn là số dương). Bản chất của luồng f_P là đẩy Δ đơn vị luồng từ s tới t trên mạng dọc theo đường P .

Ràng buộc về sức chứa khá hiển nhiên:

$\forall e \in P$: Vì Δ là dư lượng của đường P nên $0 < \Delta \leq c_f(e)$:

$$0 < f_P(e) = \Delta \leq c_f(e)$$

$\forall e \notin P: f_P(e) = 0$ nên cũng thỏa mãn ràng buộc về sức chứa.

Ràng buộc về tính bảo toàn: $\forall u \notin \{s, t\}$ nằm trên đường P thì trên đường P chỉ có một cung e_1 đi vào u và chỉ một cung e_2 ra khỏi u , hai cung này đều mang luồng $f_P(\cdot)$ bằng Δ , những cung khác đi ra hay đi vào u đều mang luồng $f_P(\cdot) = 0$, vậy

$$f_P(u, V) = f_P(e_2) = f_P(e_1) = f_P(V, u)$$

Riêng với đỉnh s , chỉ có một cung mang luồng $f_P(\cdot) = \Delta$ đi ra khỏi s , cung này là cung đầu tiên trên đường P . Những cung khác đi ra hay đi vào s đều có luồng $f_P(\cdot) = 0$, từ đó suy ra $|f_P| = \Delta$ ■

Định lý 1-5 và Định lý 1-4 (Tổng luồng) cho ta một hệ quả sau:

Hệ quả 1-6

Cho f là một luồng trên mạng $G = (V, E, c, s, t)$ và P là một đường tăng luồng trên G_f . Khi đó $f \uparrow f_P$ là một luồng mới trên G với giá trị luồng:

$$|f \uparrow f_P| = |f| + |f_P|$$

1.2.5. Tính đúng của thuật toán

Hệ quả 1-6 cho phép viết lại sơ đồ thuật toán Ford-Fulkerson như sau:

```
1 f = « Một luồng bất kỳ »;  
2 while (« Tìm được đường tăng luồng P »)  
3     f = f  $\uparrow$  fP;  
4 Output  $\leftarrow$  f;
```

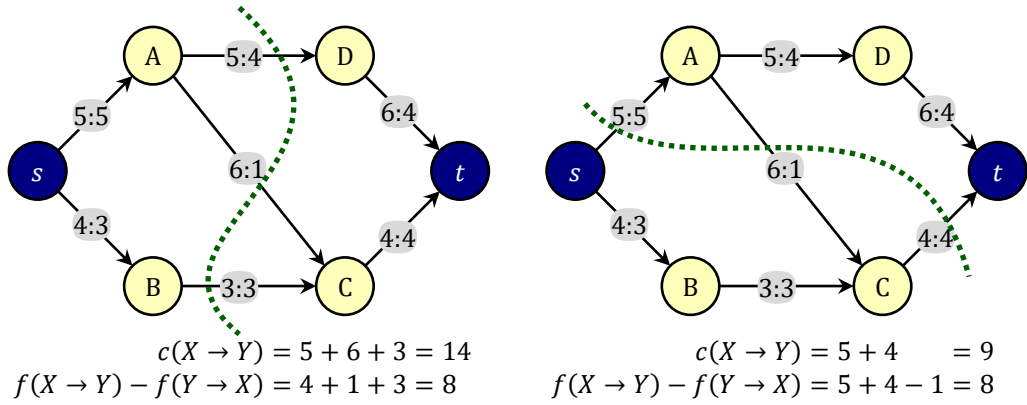
Dễ thấy thuật toán Ford-Fulkerson trả về một luồng, tức là kết quả mà thuật toán trả về thỏa mãn các tính chất của luồng. Việc chứng minh luồng đó là cực đại đã xây dựng một định lý quan trọng về mối quan hệ giữa luồng cực đại và lát cắt hẹp nhất.

Ta gọi một lát cắt (X, Y) là một cách phân hoạch tập đỉnh V làm hai tập rời nhau $(X \cup Y = V$ và $X \cap Y = \emptyset)$. Hai tập X, Y được gọi là hai phía của lát cắt. Lát cắt có $s \in X$ và $t \in Y$ được gọi là một lát cắt $s - t$.

☀ $c(X \rightarrow Y)$ được gọi là sức chứa của lát cắt

☀ $f(X \rightarrow Y) - f(Y \rightarrow X)$ được gọi là giá trị luồng đi qua lát cắt.

Hình 1-4 là ví dụ về hai lát cắt $s - t$ trên mạng



Hình 1-4. Ví dụ về sức chứa và luồng đi qua lát cắt

Bổ đề 1-7

Với f là một luồng trên mạng $G = (V, E, c, s, t)$. Khi đó giá trị luồng đi qua một lát cắt $s - t$ bất kỳ bằng $|f|$.

Chứng minh

Với (X, Y) là một lát cắt $s - t$:

$$\begin{aligned}
 f(X \rightarrow Y) - f(Y \rightarrow X) &= f(X \rightarrow Y) + f(X \rightarrow X) \\
 &\quad - f(Y \rightarrow X) - f(X \rightarrow X) \\
 &= f(X \rightarrow V) - f(V \rightarrow X) \quad (\text{Bổ đề 1-1}) \\
 &= f(s \rightarrow V) + f(X \setminus \{s\} \rightarrow V) \\
 &\quad - f(V \rightarrow s) - f(V \rightarrow X \setminus \{s\}) \\
 &= f(s \rightarrow V) - f(V \rightarrow s) \quad (\text{Bổ đề 1-2}) \\
 &= |f| \blacksquare
 \end{aligned}$$

Bổ đề 1-8

Với f là một luồng trên mạng $G = (V, E, c, s, t)$. Khi đó giá trị luồng đi qua một lát cắt $s - t$ bất kỳ không vượt quá sức chứa của lát cắt đó.

Chứng minh

Với (X, Y) là một lát cắt $s - t$:

$$\begin{aligned}
 f(X \rightarrow Y) - f(Y \rightarrow X) &\leq f(X \rightarrow Y) \\
 &= \sum_{e \in X \rightarrow Y} f(e) \\
 &\leq \sum_{e \in X \rightarrow Y} c(e) \\
 &= c(X \rightarrow Y) \blacksquare
 \end{aligned}$$

Định lý 1-9 (Mối quan hệ giữa luồng cực đại, đường tăng luồng và lát cắt hẹp nhất)

Cho f là một luồng trên mạng $G = (V, E, c, s, t)$, khi đó ba mệnh đề sau là tương đương:

- f là luồng cực đại trên mạng G .
- Mạng dư lượng G_f không có đường tăng luồng.
- Tồn tại (X, Y) là một lát cắt $s - t$ để $f(X \rightarrow Y) - f(Y \rightarrow X) = c(X \rightarrow Y)$

Chứng minh

"a $\Rightarrow$ b"

Giả sử phản chứng rằng mạng dư lượng G_f có đường tăng luồng P thì $f \uparrow f_P$ cũng là một luồng trên G với giá trị luồng lớn hơn f , trái giả thiết f là luồng cực đại trên mạng.

“b $\Rightarrow$ c”

Nếu G_f không tồn tại đường tăng luồng thì ta đặt X là tập các đỉnh đến được từ s bằng một đường dư và Y là tập các đỉnh còn lại:

$$\begin{cases} X = \{v \in V : \exists \text{ đường dư } s \rightsquigarrow v \text{ trên } G_f\} \\ Y = V \setminus X \end{cases}$$

Rõ ràng $X \cap Y = \emptyset$, $X \cup Y = V$ và $s \in X$, $s \notin Y$ nên (X, Y) là một lát cắt $s - t$.

Trước hết ta chỉ ra rằng trên mạng dư lượng, mọi cung $e = (u, v) \in X \rightarrow Y$ phải là cung bão hòa. Thật vậy, nếu e là cung dư thì từ s sẽ tới được v bằng một đường dư $s \rightsquigarrow u \rightarrow v$ trên G_f , tức là v thuộc cả X và Y . Mâu thuẫn.

Với $\forall e = (u, v) \in X \rightarrow Y$, chắc chắn $f(e) = c(e)$ vì nếu không e sẽ là cung dư nối từ X sang Y trên G_f .

Với $\forall e = (u, v) \in Y \rightarrow X$, chắc chắn $f(e) = 0$ vì nếu không thì $-e$ sẽ là cung dư nối từ X sang Y trên G_f .

Vậy thì luồng đi qua lát cắt (X, Y) bằng:

$$\begin{aligned} f(X \rightarrow Y) - f(Y \rightarrow X) &= \sum_{e \in X \rightarrow Y} f(e) - \sum_{e \in Y \rightarrow X} f(e) \\ &= \sum_{e \in X \rightarrow Y} c(e) - \sum_{e \in Y \rightarrow X} 0 \\ &= c(X \rightarrow Y) \end{aligned}$$

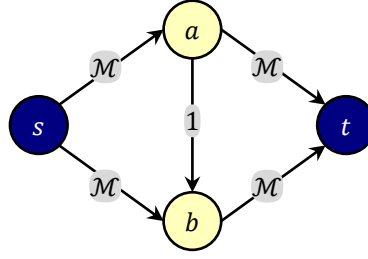
“c $\Rightarrow$ a”

Bổ đề 1-7 cho biết $|f|$ bằng giá trị luồng đi qua một lát cắt $s - t$ bất kỳ. Bổ đề 1-8 khẳng định giá trị luồng đi qua một lát cắt $s - t$ bất kỳ không thể vượt quá sức chứa của một lát cắt đó. Nếu tồn tại một lát cắt $s - t$ mà giá trị luồng đi qua lát cắt đúng bằng sức chứa của lát cắt thì luồng đó chắc chắn phải là luồng cực đại.

Lát cắt $s - t$ có lưu lượng nhỏ nhất (bằng giá trị luồng cực đại trên mạng) gọi là *Lát cắt $s - t$ hẹp nhất* của mạng G .

1.2.6. Tính dừng của thuật toán

Thuật toán Ford-Fulkerson có thời gian thực hiện phụ thuộc vào thuật toán tìm đường tăng luồng tại mỗi bước. Có thể chỉ ra được ví dụ mà nếu dùng DFS để tìm đường tăng luồng thì thời gian thực hiện giải thuật không bị chặn bởi một hàm đa thức của số đỉnh và số cạnh. Thêm nữa, nếu sức chứa của các cung là số thực, người ta còn chỉ ra được ví dụ mà với thuật toán tìm đường tăng luồng không tốt, giá trị luồng sau mỗi bước vẫn tăng nhưng **không bao giờ đạt luồng cực đại**. Tức là nếu có thể cài đặt phép tính toán số thực với độ chính xác tuyệt đối, chương trình cài đặt thuật toán sẽ chạy mãi không dừng.



Hình 1-5. Mạng với 4 đỉnh (s phát, t thu), nhãn ghi trên cung là sức chứa với M là một số rất lớn. Khi đó thuật toán Ford-Fulkerson có thể mất $2M$ lần tìm đường tăng luồng nếu luân phiên chọn hai đường $\langle s, a, b, t \rangle$ và $\langle s, b, a, t \rangle$ làm đường tăng luồng, mỗi lần tăng giá trị luồng lên 1 đơn vị

Chính vì vậy, đôi khi người ta dùng từ “phương pháp” Ford-Fulkerson để chỉ cách làm chung, còn từ “thuật toán” được dùng khi phương pháp Ford-Fulkerson được trình bày kèm một thuật toán tìm đường tăng luồng cụ thể. Ví dụ phương pháp Ford-Fulkerson cài đặt với thuật toán tìm đường tăng luồng bằng BFS như trên được gọi là thuật toán Edmonds-Karp. Tính dừng của thuật toán Edmonds-Karp sẽ được chỉ ra khi đánh giá thời gian thực hiện giải thuật.

Giả sử ta đang có luồng f , với $\forall v \in V$, ký hiệu $\delta_f(v)$ là độ dài đường dư ngắn nhất từ s tới v trên G_f (tính bằng số cung đi qua). Nếu không tồn tại đường dư $s \rightsquigarrow v$, coi như $\delta_f(v) = +\infty$.

Bổ đề 1-10

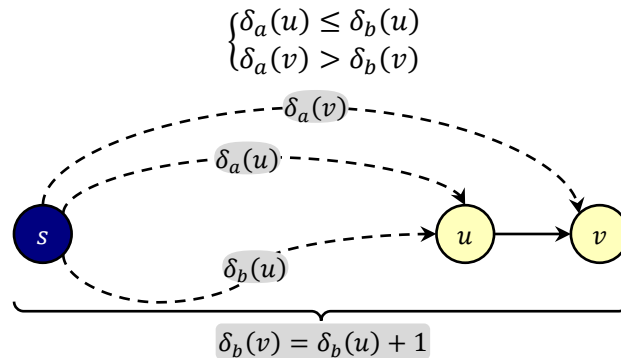
Nếu ta thực hiện thuật toán Edmonds-Karp trên mạng $G = (V, E, c, s, t)$. Khi đó với mọi đỉnh $v \in V$, độ dài đường dư ngắn nhất từ s tới v không giảm sau mỗi bước tăng luồng.

Chứng minh

Giả sử a là một luồng trên mạng và sau khi tăng luồng ta thu được luồng b , ta sẽ chỉ ra rằng $\delta_a(v) \leq \delta_b(v)$ với $\forall v \in V$.

Khi $v = s$, rõ ràng $\delta_a(s) = \delta_b(s) = 0$.

Giả sử phản chứng rằng tồn tại một đỉnh $v \neq s$ mà $\delta_a(v) > \delta_b(v)$. Nếu có nhiều đỉnh v như vậy ta chọn đỉnh v có $\delta_b(v)$ nhỏ nhất. Gọi (u, v) là cung cuối cùng của đường dư ngắn nhất $s \rightsquigarrow v$ trên G_b . Từ giả thiết phản chứng và cách chọn u, v :



Nếu (u, v) cũng là cung dư trên G_a thì:

$$\begin{aligned} \delta_a(v) &\leq \delta_a(u) + 1 \text{ (bất đẳng thức tam giác)} \\ &\leq \delta_b(u) + 1 \text{ (do cách chọn } u) \\ &= \delta_b(v) \quad \text{(mâu thuẫn)} \end{aligned}$$

Để ý rằng khi tăng luồng a thành luồng b thì dư lượng các cung dọc theo đường tăng luồng giảm đi, dư lượng các cung đối của chúng tăng lên, còn dư lượng các cung khác không đổi. Để (u, v) là cung bão hòa trên G_a nhưng lại là cung dư trên G_b thì cung đối (v, u) phải là một cung trên đường tăng luồng, tức là sẽ có đường dư ngắn nhất $s \rightsquigarrow v \rightarrow u$ trên mạng dư lượng G_a . Suy ra:

$$\begin{aligned}\delta_a(v) &= \delta_a(u) - 1, ((v, u) \text{ thuộc đường tăng luồng trên } G_a) \\ &\leq \delta_b(u) - 1, (\text{do cách chọn } u) \\ &= \delta_b(v) - 2, ((u, v) \text{ thuộc đường tăng luồng trên } G_b) \\ &< \delta_b(v)\end{aligned}$$

Mâu thuẫn với giả thiết khoảng cách từ s tới v phải giảm đi sau khi tăng luồng. ĐPCM ■

Bổ đề 1-11

Số lượt tăng luồng được sử dụng trong thuật toán Edmonds-Karp là $O(nm)$.

Chứng minh

Ta chia quá trình thực hiện thuật toán Edmonds-Karp thành các pha. Mỗi pha tìm một đường tăng luồng P và tăng luồng thêm một giá trị bằng Δ . Theo thuật toán, Δ phải bằng sức chứa nhỏ nhất của một cung dư e nào đó trên đường P :

$$\exists e \in P: \Delta = c_f(e)$$

Khi tăng luồng dọc trên đường P thì cung e sẽ bị giảm dư lượng đi Δ , dư lượng của e trở thành 0 tức là e trở thành cung bão hòa. Những cung dư trở nên bão hòa sau khi tăng luồng gọi là *cung tới hạn (critical edge)* tại mỗi pha. Mỗi pha có ít nhất một cung tới hạn.

Ta đánh giá xem mỗi cung của mạng có thể trở thành cung tới hạn bao nhiêu lần. Với một cung $e = (u, v)$, xét một pha làm e trở thành cung tới hạn, nếu luồng trước pha đó là a thì do e thuộc đường tăng luồng trên G_a :

$$\delta_a(u) + 1 = \delta_a(v)$$

Để cung e trở thành cung tới hạn một lần nữa thì trước đó phải có một pha biến e thành cung dư. Nếu luồng trước pha này là b thì $-e = (v, u)$ phải thuộc đường tăng luồng trên G_b :

$$\delta_b(v) + 1 = \delta_b(u)$$

Bổ đề 1-10 đã chứng minh rằng độ dài đường dư ngắn nhất từ s tới v trên mạng dư lượng không giảm đi sau mỗi pha. Suy ra:

$$\begin{aligned}\delta_b(u) &= \delta_b(v) + 1 \\ &\geq \delta_a(v) + 1 \text{ (Bổ đề 1-10)} \\ &\geq \delta_a(u) + 2\end{aligned}$$

Như vậy ngay trước pha biến $e = (u, v)$ thành cung tới hạn một lần nữa, đường dư ngắn nhất $s \rightsquigarrow u$ tồn tại và có độ dài tăng lên ít nhất 2 đơn vị. Độ dài đường dư ngắn nhất (nếu $< +\infty$) không thể vượt quá $n - 1$, nên số pha nhận e làm cung tới hạn là một đại lượng $O(n)$

Tổng hợp lại, ta có:

- ☀ Mạng có m cung,
- ☀ Mỗi pha có ít nhất một cung tới hạn,
- ☀ Một cung có thể trở thành tới hạn trong $O(n)$ pha

Vậy tổng số pha được thực hiện trong thuật toán Edmonds-Karp là một đại lượng $O(n * m)$ ■

Định lý 1-12 (Độ phức tạp của thuật toán Edmonds-Karp)

Thuật toán Edmonds-Karp tìm được luồng cực đại trên mạng $G = (V, E, c, s, t)$ trong thời gian $O(n * m^2)$.

Chứng minh

Bổ đề 1-11 đã chứng minh rằng thuật toán Edmonds-Karp cần thực hiện $O(n * m)$ lượt tăng luồng. Tại mỗi lượt thuật toán mất thời gian $O(m)$ để tìm đường tăng luồng bằng BFS và tăng luồng dọc đường này. Suy ra thời gian thực hiện giải thuật Edmonds-Karp là $O(n * m^2)$ ■

Định lý 1-13 (Định lý về tính nguyên)

Nếu sức chứa trên các cung là số nguyên và luồng khởi tạo trên các cung là số nguyên thì thuật toán Ford-Fulkerson tìm được luồng cực đại với giá trị luồng trên các cung là số nguyên

Chứng minh

Tại mỗi bước lặp của thuật toán Ford-Fulkerson, nếu sức chứa và luồng trên các cung là số nguyên thì dư lượng của các cung cũng phải là số nguyên, từ đó dư lượng của đường tăng luồng phải số nguyên dương. Việc tăng giảm luồng trên các cung lên một số nguyên dương không làm mất tính nguyên của luồng trên cung.

Trong trường hợp này, nếu ta khởi tạo luồng 0 và thực hiện thuật toán Ford-Fulkerson, bất kể thuật toán tìm đường tăng luồng là BFS hay DFS, luồng sau khi tăng sẽ có giá trị hơn trước ít nhất 1 đơn vị. Khi ấy thời gian thực hiện giải thuật có thể đánh giá bằng $O(|f| * m)$ với $|f|$ là giá trị luồng cực đại ■

1.3. Thuật toán Dinic

Thuật toán Ford-Fulkerson không những là một cách tiếp cận thông minh mà việc chứng minh tính đúng đắn của nó cho ta nhiều kết quả thú vị về mối liên hệ giữa luồng cực đại và lát cắt hẹp nhất. Tuy vậy với những đồ thị kích thước rất lớn thì tốc độ của chương trình tương đối chậm.

Dinitz phát kiến ra một kỹ thuật mới gọi là *khóa luồng (blocking flow)* [3], một kỹ thuật rất tốt để cải thiện tốc độ của phương pháp Ford-Fulkerson. Kỹ thuật này được ông tìm ra

trong một bài tập của lớp học thuật toán và vào thời điểm đó ông không hề biết rằng mình là người đầu tiên tìm ra một thuật toán đa thức giải bài toán luồng cực đại*.

Ý tưởng của thuật toán Dinic là tại mỗi bước, xây dựng một cơ chế phân tầng các đỉnh dựa trên độ dài đường đi ngắn nhất tới đỉnh thu trên mạng dư lượng, sau đó tìm một luồng trên mạng dư lượng để hợp vào luồng chính. Khác với thuật toán Edmonds-Karp (trong đó luồng trên mạng dư lượng chỉ dựa vào 1 đường tăng luồng), thuật toán Dinic tăng luồng theo tất cả các đường tăng luồng ngắn nhất bằng cách “khóa luồng” hay “chặn dòng”: chỉ cho phép luồng đổ từ đỉnh tầng trên xuống đỉnh tầng liền dưới.

1.3.1. Mạng phân tầng và cơ chế tăng luồng

Cho f là một luồng trên mạng $G = (V, E, c, s, t)$. Với mỗi đỉnh u , gọi $h[u]$ là độ dài đường dư ngắn nhất từ u tới t trên mạng dư lượng. Nếu u không tới được t bằng đường dư, ta coi như $h[u] = +\infty$ ứng với việc đỉnh u không được phân tầng.

Việc tính các giá trị $h[.]$ có thể thực hiện bằng thuật toán BFS với đỉnh xuất phát là t , đi ngược chiều các cung. Nhân $h[u]$ được gọi là tầng của u .

Nếu s không đến được t (s không được phân tầng) thì f là luồng cực đại. Nếu không, thuật toán lần lượt tìm các đường tăng luồng ngắn nhất từ s tới t (qua đúng $h[s]$ cung dư) và tăng luồng dọc trên các đường tăng luồng này. Pha này được xử lý khéo léo bằng thuật toán DFS trên mạng dư lượng đã phân tầng.

Khi không còn đường tăng luồng độ dài $h[s]$ nữa, thuật toán Dinic sẽ phân tầng lại và tiếp tục quá trình tương tự...

Trong khâu tìm đường đi từ s tới t trên mạng đã phân tầng, những cung dư nối từ một tầng xuống tầng liền dưới được gọi là cung được thừa nhận (*admissible*), những cung khác được coi là bị khóa (*blocked*). Ý tưởng của thuật toán Dinic sẽ được trình bày kỹ hơn trong phần cài đặt

1.3.2. Cài đặt

• Tổ chức dữ liệu

Thuật toán Dinic sẽ được cài đặt với khuôn dạng Input/Output như trong chương trình cài đặt thuật toán Edmonds-Karp.

Việc tổ chức dữ liệu trước hết kế thừa hoàn toàn thuật toán Edmonds-Karp, ta tóm tắt một số ý chính:

* Thuật toán của Dinic được công bố năm 1970 trên một tạp chí bằng tiếng Nga. Năm 1974, Shimon Even và Alon Itai đọc được bài báo và bị hấp dẫn với ý tưởng này nên đưa vào một bài giảng “Thuật toán Dinic” nhưng lại đánh máy sai tên tác giả khi phổ biến nó. Tuy nhiên hai tác giả này cũng góp phần vào thuật toán bằng cách kết hợp giữa BFS và DFS trở thành phiên bản hiện tại của thuật toán này. Hiện nay ta có thể gọi dưới tên Dinic hay Dinic đều được. Mặc dù năm 2006 Dinic có viết một bài báo phân định rõ ràng ý tưởng của mình và những bổ sung của Even nhưng thuật toán vẫn được gọi dưới cái tên Dinic.

Thuật toán Dinic (1970) có trước thuật toán Edmonds-Karp (1972), mục đích của Dinic chỉ là tìm một thuật toán đa thức cho phương pháp Ford-Fulkerson chứ không phải cải tiến thuật toán Edmonds-Karp. Người ta nhận ra kỹ thuật khóa luồng có thể tăng tốc thuật toán Edmonds-Karp chính là qua bài giảng của Even (1974).

- ☀ Mỗi cung chứa thông tin về hai đỉnh đầu cuối: x, y và dư lượng c_f
- ☀ m cung thật và m cung giả được lưu trữ trong danh sách $e[0 \dots 2m]$, cung thật ở vị trí chẵn, cung giả ở vị trí lẻ, cung đối của $e[i]$ là cung $e[i \wedge 1]$
- ☀ Mỗi đỉnh u có một danh sách $a[u]$ chứa chỉ số các cung đi ra khỏi u

• Kỹ thuật phân tầng

Ta cần tính các giá trị $h[v]$ là độ dài đường dư ngắn nhất từ v tới t (tính bằng số cạnh), những đỉnh không có đường dư đến v có $h[v] = n$. Điều này có thể thực hiện bằng cách đảo chiều các cung rồi thực hiện thuật toán BFS từ t .

Tuy nhiên khi cài đặt ta không cần đảo chiều các cung, với mỗi đỉnh u , để quét tất cả các cung đi vào u , ta có thể quét danh sách $a[u]$ để duyệt các cung e ra khỏi u , rồi thay vì xử lý cung đó ta sẽ xử lý cung đối.

• Đẩy luồng dọc các đường tăng luồng ngắn nhất

Dễ thấy rằng một đường tăng luồng ngắn nhất (độ dài $h[s]$) sẽ chỉ đi qua các cung được thừa nhận (là những cung dư (u, v) có $h[u] = h[v] + 1$).

Phép tăng luồng dọc các đường tăng luồng ngắn nhất được thực hiện bằng thuật toán tìm kiếm theo chiều sâu (DFS), ở đây là mô hình DFS theo cung chứ không phải DFS theo đỉnh: Bắt đầu với các cung dư nối từ s (ở tầng $h[s]$) sang tầng $h[s] - 1$, thuật toán duyệt từ một cung sang cung dư nối tiếp ở tầng dưới cho tới khi tới được t .

Trong quá trình đi từ tầng $h[s]$ xuống tầng 0, thuật toán mang một lượng luồng đi theo. Do ràng buộc về sức chứa, có thể một lượng luồng nào đó bị ứ đọng tại đỉnh (gọi là lượng tràn) mà không phát tán được tới t . Khi quá trình đệ quy của DFS lùi về, lượng tràn tại đỉnh sẽ được thu hồi lên đỉnh ở tầng liền trên để cố gắng phát tán theo hướng khác... Chi tiết kỹ thuật được giải thích như dưới đây:

Hàm $DFS(u, flowIn)$ có ý nghĩa: Đổ một lượng luồng $flowIn$ vào đỉnh u sau đó phát tán lượng luồng này xuống các đỉnh ở tầng dưới thì cuối cùng có bao nhiêu đơn vị luồng đến được t . Lượng luồng đến được t sẽ trả về trong kết quả hàm.

Phép tăng luồng trên mạng phân tầng thực hiện bằng lời gọi $DFS(s, \mathcal{M})$ với $\mathcal{M}$ là một hằng số đủ lớn, đảm bảo không nhỏ hơn giá trị luồng cực đại.

Cơ chế thực hiện của hàm $DFS(u, flowIn)$:

Nếu $u = t$, hàm trả về $flowIn$ (luồng phát tán tới t bao nhiêu cũng hết).

Nếu $u \neq t$, đặt $excess = flowIn$ là lượng tràn muốn phát tán tới t , sau đó tìm cách tháo lượng tràn này xuống các đỉnh tầng dưới: Xét tất cả các cung dư (u, v) đi xuống tầng liền dưới, với mỗi cung đó:

- ☀ Tính lượng luồng tối đa có thể đẩy theo cung (u, v) trên mạng dư lượng:

$$charged = \min(excess, c_f(u, v));$$

- ☀ Cho v nhận vào lượng luồng bằng $charged$ và phát tán xuống các tầng dưới, tính lượng luồng tới được t bằng đệ quy:

$$discharged = DFS(v, charged);$$

- ☀ Theo mô tả về cách thức hoạt động của hàm $DFS(v, charged)$, đã có $discharged$ đơn vị luồng phát tán từ v tới t . Bây giờ, ta đẩy đúng $discharged$ đơn vị luồng qua cung (u, v) , tức là cấp cho v đúng $discharged$ đơn vị luồng không thừa không thiếu, giữ tính bảo toàn luồng tại v .
- ☀ Cập nhật lượng tràn còn lại ($excess -= discharged$) trước khi xét một cung (u, v') khác để phát tán theo hướng mới. Nếu $excess$ trở thành 0, hàm sẽ thoát ngay.

Hàm trả về $flowIn - excess$ là lượng tràn đã chuyển được tới t .

• Con trở tới cung còn hiệu lực

Để thuật toán hiệu quả hơn, mỗi đỉnh u sẽ được kèm theo một con trỏ $ptr[u]$ tới cung đang xét trong danh sách $a[u]$. Ngay sau khi phân tầng, các con trỏ $ptr[u]$ được đặt vào đầu danh sách $a[u]$. Trong thuật toán DFS, khi xét danh sách các cung đi ra khỏi u ta sẽ xét từ con trỏ $ptr[u]$, nếu cung này là cung bão hòa hay cung không nối tới đỉnh ở tầng liền dưới, con trỏ $ptr[u]$ sẽ được dịch sang cung kế tiếp. Bởi thuật toán DFS ở đây có thể thăm một đỉnh nhiều lần, con trỏ $ptr[.]$ cho phép ta không cần phải duyệt lại từ đầu danh sách liên thuộc chứa những cung không được thừa nhận.

DINIC.cpp Thuật toán DINIC

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5  using lli = long long;
6  const int MAX_N = 1e4;
7  const int MAX_M = 1e5;
8  const int MAX_C = 1e9;
9  const lli MAX_FLOW_VAL = lli(MAX_M) * MAX_C; //Giá trị không nhỏ hơn luồng cực đại
10
11 struct TEdge //Cấu trúc một cung
12 {
13     int x, y; //Hai đỉnh đầu nút
14     int cf;    //dư lượng
15 };
16
17 int n, m, s, t;
18 TEdge e[MAX_M * 2];
19 vector<int> a[MAX_N]; //Danh sách liên thuộc
20 vector<int>::const_iterator ptr[MAX_N]; //ptr[u] là cung đang xét trong a[u]
21 int h[MAX_N];
22
23 void Enter()
24 {
25     cin >> n >> m >> s >> t;
26     for (int i = 0; i < m * 2; i += 2)
27     {
28         int u, v, cap;
29         cin >> u >> v >> cap; //Đọc một cung (u, v) và sức chứa cap
30         e[i] = {u, v, cap}; //Thêm cung (u, v) với dư lượng cap
31         e[i + 1] = {v, u, 0}; //Thêm cung đối (v, u) với dư lượng 0
32         a[u].push_back(i);
33         a[v].push_back(i + 1);
34     }
35 }
36

```

```

37 bool BFS() //Phân tầng bằng BFS, trả về true nếu có đường tăng luồng
38 {
39     queue<int> q({t}); //hàng đợi dùng cho BFS, ban đầu chỉ có đỉnh t
40     fill(h, h + n, n); //Nhãn h[ ] == n cho biết đỉnh chưa phân tầng
41     h[t] = 0; //Riêng đỉnh thu có h[t] = 0
42     while (!q.empty())
43     {
44         int u = q.front(); q.pop();
45         for (int i: a[u]) //Quét các cung e[i] = (u, v) ra khỏi u
46         {
47             int v = e[i].y; //cung đối e[i ^ 1] = (v, u) là cung đi vào u
48             if (e[i ^ 1].cf == 0 || h[v] < n) //cung đối bão hòa hoặc v đã phân tầng
49                 continue; //bỏ qua
50             h[v] = h[u] + 1; //Phân tầng cho đỉnh v, cũng là đánh dấu h[v] < n
51             if (v == s) //s đã được phân tầng
52                 return true; //dừng ngay và trả về true
53             q.push(v);
54         }
55     }
56     return false; //không có đường tăng luồng
57 }
58
59 void IncFlow(int i, int Delta) //Đẩy Delta đơn vị luồng theo cung e[i]
60 {
61     e[i].cf -= Delta; //Dư lượng cung e[i] giảm đi Delta
62     e[i ^ 1].cf += Delta; //Dư lượng cung đối tăng lên Delta
63 }
64
65 //Nếu phát tán flowIn luồng từ u thì tới t được bao nhiêu?
66 lli DFS(int u, lli flowIn) //Thuật toán DFS trên mạng phân tầng
67 {
68     if (u == t)
69         return flowIn;
70     lli excess = flowIn; //Lượng tràn tại u cần phát tán
71     for (; ptr[u] != a[u].end(); ++ptr[u]) //Xét các cung từ ptr[u] trở đi
72     {
73         int i = *ptr[u];
74         int v = e[i].y;
75         if (e[i].cf == 0 || h[v] != h[u] - 1)
76             continue; //Bỏ qua cung không được thừa nhận
77         lli charged = min(excess, (lli)e[i].cf); //Lượng luồng tối đa muốn đẩy qua e[i]
78         lli discharged = DFS(v, charged); //Lượng luồng phát tán được từ v tới t
79         IncFlow(i, discharged); //Thêm discharged luồng theo cung e[i] từ u tới v
80         excess -= discharged; //Lượng tràn tại u giảm đi discharged
81         if (excess == 0) //Hết lượng tràn,
82             break; //dừng ngay
83     }
84     return flowIn - excess; //Trả về lượng luồng phát tán được tới t
85 }
86
87 void BlockingFlow() //Tăng luồng theo tất cả đường tăng luồng ngắn nhất
88 {
89     for (int u = 0; u < n; ++u)
90         ptr[u] = a[u].begin(); //Đặt con trỏ ptr[u] vào đầu danh sách a[u]
91     DFS(s, MAX_FLOW_VAL); //Thủ phát tán lượng luồng đủ lớn từ s
92 }
93

```

```

94 void Print() //In kết quả, không khác gì trước
95 {
96     lli flowVal = 0;
97     for (int i: a[s]) //Quét các cung ra khỏi s
98         if (i % 2 == 0) //Gặp cung thật ra khỏi s
99             flowVal += e[i ^ 1].cf; //Cộng luồng ra khỏi s
100         else //Gặp cung giả ra khỏi s, cung đối là cung thật đi vào s
101             flowVal -= e[i].cf; //Trừ luồng đi vào s
102     cout << "Value of flow: " << flowVal << '\n';
103     //Chỉ ra sức chứa và luồng trên các cung đã cho bằng thông tin về dư lượng
104     for (int i = 0; i < m * 2; i += 2) //Cung thật mang chỉ số chẵn trong e
105         cout << "e[" << i / 2 << "] = "
106             << "(" << e[i].x << ", " << e[i].y << ")": "
107             << "c = " << e[i].cf + e[i ^ 1].cf << ", " //c(e) = cf(e) + cf(-e)
108             << "f = " << e[i ^ 1].cf << '\n'; //f(e) = cf(-e)
109 }
110
111 int main()
112 {
113     Enter(); //Nhập dữ liệu
114     while (BFS())
115         BlockingFlow();
116     Print(); //In kết quả
117 }

```

1.3.3. Thời gian thực hiện giải thuật

Xét vòng lặp chính của thuật toán, tại mỗi bước lặp, độ dài đường dư ngắn nhất từ s tới t là $h[s]$. Sau khi gọi hàm `BlockingFlow()` sẽ không còn đường tăng luồng độ dài $h[s]$ nữa. Tương tự như trong chứng minh thuật toán Edmonds-Karp, điều này cho thấy giá trị $h[s]$ tăng ngặt sau mỗi lượt lặp, và vì $h[s] \leq n - 1$, số lần gọi hàm `BlockingFlow()` là một đại lượng $O(n)$.

Trong mỗi bước lặp, thuật toán BFS để phân tầng có thời gian thực hiện $O(n + m)$.

Xét số lần gọi hàm DFS một lần gọi `BlockingFlow()`, giả sử hàm $DFS(u, \square)$ gọi hàm $DFS(v, \square)$ sau đó tính lại lượng tràn (*Excess*) mới...

- ☀ Nếu giá trị *Excess* trở thành 0, hàm $DFS(u, \square)$ thoát, quá trình DFS lùi lại.
- ☀ Nếu giá trị *Excess* vẫn > 0 , con trỏ $ptr[u]$ dịch đi một vị trí đồng nghĩa với việc cung (u, v) bị loại và không bao giờ bị xét đến trong lần gọi `BlockingFlow()` này nữa.

Quá trình DFS tìm đường đi từ s tới t qua tất cả các tầng của mạng từ tầng $h[s]$ tới tầng 0. Vì vậy dây chuyền đệ quy không thể lùi về liên tục quá $h[s]$ bước, tức là trong $h[s]$ lần hàm $DFS(\square, \square)$ thoát, chắc chắn có một cung bị loại. Số cung là m suy ra số lần thoát hàm $DFS(\square, \square)$ (cũng là số lần gọi hàm) là $O(m * h[s]) = O(n * m)$.

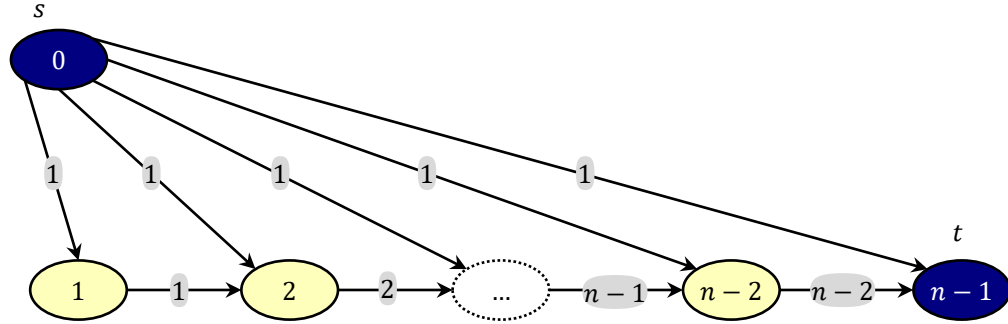
Còn lại là tổng thời gian duyệt danh sách liên thuộc dựa vào con trỏ $ptr[.]$, mỗi lần con trỏ $ptr[.]$ được xét sẽ là một lệnh thoát hàm hoặc một phép dịch con trỏ $ptr[.]$: Vậy số lần xét con trỏ $ptr[.]$ sẽ là $O(n * m + m) = O(n * m)$

Vậy hàm `BlockingFlow()` thực hiện trong thời gian $O(n * m)$ và thuật toán Dinic có thời gian thực hiện là $O(n^2 * m)$.

1.4. Thuật toán đẩy tiền luồng

Mặc dù thuật toán Dinic có tốc độ tốt trong thực tế, cũng có trường hợp xấu khiến cho tại mỗi pha `BlockingFlow()` thuật toán chỉ tìm được một đường tăng luồng duy nhất, nghĩa là

cách hoạt động của thuật toán Dinic không khác gì thuật toán Edmonds-Karp, lại mất thêm thời gian xây dựng mạng phân tầng.



Hình 1-6. Ví dụ xấu của thuật toán Dinic, số ghi trên cung là sức chứa, cũng là luồng trên cung ứng với luồng cực đại

Trong phần này ta sẽ trình bày một lớp các thuật toán nhanh nhất cho tới nay để giải bài toán luồng cực đại, tên chung của các thuật toán này là thuật toán *đẩy tiền luồng* (*preflow-push hay push-relabel*).

Ta nhìn nhận mạng như một hệ thống đường ống dẫn nước từ với đỉnh phát s tới đỉnh thu t , các cung là các đường ống, sức chứa là lưu lượng nước có thể tải qua đường ống trong một đơn vị thời gian. Nước chảy theo nguyên tắc từ chỗ cao về chỗ thấp. Khởi đầu với một lượng nước lớn phát ra từ s tới các đỉnh lân cận, nếu có cách chuyển tiếp lượng nước đó sang đỉnh khác thấp hơn thì quá trình tiếp tục, nếu không thì có hiện tượng “tràn” xảy ra tại một đỉnh, ta “dâng cao” đỉnh tràn để tiếp tục tháo nước ra khỏi đỉnh (có thể đổ ngược về s). Cứ tiếp tục quá trình như vậy cho tới khi không còn hiện tượng tràn ở bất cứ điểm nào.

1.4.1. Tiền luồng

Cho mạng $G = (V, E, c, s, t)$. Một *tiền luồng* (*preflow*) trên G là một hàm:

$$\begin{aligned} f: E &\rightarrow \mathbb{R}_{\geq 0} \\ e &\mapsto f(e) \end{aligned}$$

gán cho mỗi cung $e \in E$ một số thực $f(e)$ thỏa mãn hai ràng buộc:

- ☀ Ràng buộc về sức chứa (*capacity constraint*): tiền luồng trên mỗi cung không được vượt quá sức chứa của cung đó: $\forall e \in E: f(e) \leq c(e)$.
- ☀ Ràng buộc về tính dư: Với mọi đỉnh u không phải đỉnh phát và không phải đỉnh thu, tổng tiền luồng trên các cung đi vào u không nhỏ hơn tổng tiền luồng trên các cung ra khỏi u : $\forall u \in V - \{s, t\}: f(V \rightarrow u) \geq f(u \rightarrow V)$.

Hiệu số tổng tiền luồng trên các cung đi vào u trừ tổng tiền luồng trên các cung ra khỏi u được gọi là lượng tràn tại u , ký hiệu $excess[u]$:

$$excess[u] = f(V \rightarrow u) - f(u \rightarrow V)$$

Ràng buộc về tính dư có thể hiểu là: lượng tràn tại các đỉnh không phải đỉnh phát cũng không phải đỉnh thu là một số không âm: $\forall u \in V \setminus \{s, t\}: excess[u] \geq 0$

Đỉnh $v \in V \setminus \{s, t\}$ gọi là *đỉnh tràn* (*overflowing vertex*) nếu $excess[v] > 0$. Khái niệm đỉnh tràn chỉ có nghĩa với các đỉnh không phải đỉnh phát cũng không phải đỉnh thu.

Một luồng bất kỳ luôn là tiền luồng nhưng một tiền luồng chỉ là luồng khi trên mạng không có đỉnh tràn. Ta cũng có khái niệm mạng dư lượng, cung dư, đường dư... ứng với tiền luồng tương tự như định nghĩa đối với luồng. Lượng tràn tại s và t không có ý nghĩa trong thuật toán, tuy nhiên ta có thể sử dụng chúng để tính nhanh giá trị tiền luồng (khi nó trở thành một luồng):

$$|f| = excess[t] = -excess[s]$$

1.4.2. Hàm độ cao ứng với tiền luồng

Cho f là một tiền luồng trên mạng $G = (V, E, c, s, t)$. Ta gọi $h: V \rightarrow \mathbb{N}$ là một hàm độ cao thích ứng với f nếu h gán cho mỗi đỉnh $v \in V$ một số tự nhiên $h[v]$ thỏa mãn ba điều kiện gọi là **ràng buộc độ cao**:

$$\begin{cases} h[s] = n \\ h[t] = 0 \\ h[u] \leq h[v] + 1, \text{ với mọi cung dư } (u, v) \in E_f \end{cases}$$

Giải thích về ràng buộc độ cao:

Đỉnh phát luôn có độ cao n và đỉnh thu luôn có độ cao 0. Cung dư từ một đỉnh có thể tùy ý nối tới đỉnh cao hơn (hoặc bằng) nhưng nếu cung dư nối tới đỉnh thấp hơn thì chỉ thấp hơn đúng 1 đơn vị. Trong các thuật toán trước đây, đường tăng luồng là đường đi đơn tạo thành từ các cung dư. Nếu bổ sung điều kiện: tiền luồng chỉ đi từ đỉnh cao xuống đỉnh thấp, thì không đường tăng luồng nào có thể “chảy” được, đơn giản vì không kịp hạ độ cao từ n (đỉnh phát) về 0 (đỉnh thu).

1.4.3. Phép đẩy

Phép đẩy (*Push*) có thể thực hiện trên cung $e = (u, v) \in E_f$ nếu các điều kiện sau được thỏa mãn:

- ☀ u là đỉnh tràn: $u \notin \{s, t\}$ và $excess[u] > 0$
- ☀ e là cung dư: $c_f(e) > 0$
- ☀ u cao hơn v : $h[u] > h[v]$

Ràng buộc $h[u] > h[v]$ kết hợp với ràng buộc độ cao: $h[u] \leq h[v] + 1$ có thể viết thành $h[u] = h[v] + 1$.

Phép đẩy tính:

$$\Delta = \min(excess[u], c_f(e)),$$

rồi đẩy thêm Δ đơn vị tiền luồng theo cung e , cập nhật $excess[u]$ cũng như $excess[v]$ theo tiền luồng mới.

Chúng ta đã có sẵn hàm $\text{IncFlow}(e \in E, \Delta)$ để đẩy thêm Δ đơn vị tiền luồng theo cung e (mục 1.1.5). Ta bổ sung vào hàm $\text{IncFlow}(e = (u, v) \in E, \Delta)$ thao tác cập nhật lượng tràn tại u và v :

```

1 void IncFlow(e = (u, v) ∈ E, Δ)
2 {
3     «Thực hiện như trước»;
4     excess[u] -= Δ; //Lượng tràn tại u giảm Δ
5     excess[v] += Δ; //Lượng tràn tại v tăng Δ
6 }
```


Lệnh $\text{IncFlow}(e, \Delta)$ tăng $f(e)$ lên Δ nếu e là cung thật, giảm $f(-e)$ đi Δ nếu e là cung giả. Từ vị trí đỉnh u , dù tăng tiền luồng ra khỏi u hay giảm tiền luồng đi vào u đều làm cho lượng tràn tại u giảm đi. Tương tự như vậy, dù tăng tiền luồng đi vào v hay giảm tiền luồng ra khỏi v đều làm cho lượng tràn tại v tăng lên. Điều này giải thích cho hai lệnh cập nhật lượng tràn:

$$\begin{aligned} \text{excess}[u] &-= \Delta; \\ \text{excess}[v] &+= \Delta; \end{aligned}$$

Phép đẩy chỉ đơn giản tính giá trị Δ , rồi gọi hàm $\text{IncFlow}(e, \Delta)$:

```
1 void Push(e = (u, v))
2 {
3     Δ = min(excess(u), cf(e));
4     IncFlow(e, Δ); //Thêm Δ tiền luồng theo cung e
5 }
```

Do cách xác định Δ , phép đẩy đảm bảo tiền luồng trên cung không âm và không vượt quá sức chứa, cũng không làm lượng tràn tại đỉnh nào bị âm, vì vậy các ràng buộc của tiền luồng vẫn được thỏa mãn.

- ☀ Nếu $\Delta = c_f(e)$ thì phép $\text{Push}(e)$ sẽ làm cho $c_f(e)$ trở thành 0, ta gọi phép đẩy luồng này là *đẩy bão hòa (saturating push)*,
- ☀ Nếu $\Delta < c_f(e)$ tức là $\Delta = \text{excess}[u]$, phép đẩy này gọi là *đẩy không bão hòa (non-saturating push)*. Sau phép đẩy không bão hòa thì $\text{excess}[u] = 0$, tức là u không còn là đỉnh tràn nữa.

1.4.4. Phép nâng

Phép nâng đỉnh u : $\text{Lift}(u)$ có thể thực hiện nếu các điều kiện sau được thỏa mãn:

- ☀ u là đỉnh tràn: $(u \neq s)$, $(u \neq t)$ và $\text{excess}[u] > 0$.
- ☀ Không tồn tại cung dư để chuyển lượng tràn từ u xuống nơi thấp hơn, hay nói cách khác, mọi cung dư nối từ u đều tới nơi cao hơn hoặc bằng u :

$$\forall e = (u, v) \in E_f: h[u] \leq h[v]$$

Khi đó phép $\text{Lift}(u)$ nâng đỉnh u lên vừa đủ để nó có cung dư tới đỉnh thấp hơn. Cụ thể là đặt $h[u]$ bằng độ cao thấp nhất của một đỉnh v nối từ u bằng cung dư cộng thêm 1:

$$h[u] = \min\{h[v]: v \neq u \text{ và } (u, v) \in E_f\}$$

```
1 void Lift(u ∈ V)
2 {
3     h[u] = +∞;
4     for (∀v: (u, v) ∈ E_f)
5         h[u] = min(h[u], h[v]);
6     ++h[u];
7 }
```

Ta chỉ ra rằng nếu u là đỉnh tràn, thì phép tính $\min\{h[v]: (u, v) \in E_f\}$ được thực hiện trên một tập khác rỗng. Thật vậy, vì u là đỉnh tràn nên phải có cung thật mang tiền luồng dương đi vào u . Cung đối của nó sẽ là cung ra khỏi u với dư lượng bằng tiền luồng trên cung thật, đây là một cung dư ra khỏi u .

Cần lưu ý trong trường hợp mạng có khuyên: Nếu có khuyên $(u, u) \in E_f$ thì phép $\text{Lift}(u)$ ở trên sẽ chỉ tăng $h[u]$ lên 1, và như vậy vẫn có thể không có cung dư từ u tới đỉnh thấp hơn, trái với mục đích của phép nâng. Các khuyên không có ý nghĩa trong bài toán luồng

cực đại, chúng nên được loại bỏ trước khi thực hiện thuật toán cho đỡ mất thời gian, cũng là để tránh những rắc rối như vậy.

1.4.5. Khởi tạo

Thao tác khởi tạo `Init()` chịu trách nhiệm khởi tạo một tiền luồng và một hàm độ cao tương ứng. Một cách khởi tạo là đặt tiền luồng trên mỗi cung đi ra khỏi s đúng bằng sức chứa của cung đó còn tiền luồng trên các cung khác bằng 0. Khi đó tất cả các cung đi ra khỏi s là bão hòa:

$$f(e) = \begin{cases} c(e), & \text{nếu } e \in \{s \rightarrow V\} \\ 0, & \text{nếu } e \notin \{s \rightarrow V\} \end{cases}$$

Hàm độ cao $h: V \rightarrow \mathbb{N}$ được khởi tạo như sau:

$$h[u] = \begin{cases} n, & \text{nếu } u = s \\ 0, & \text{nếu } u \neq s \end{cases}$$

Rõ ràng mọi cung dư ứng với tiền luồng này không thể là cung đi ra khỏi s nên với mọi cung dư (u, v) thì $h[u] = 0$. Tức là $h[u] \leq h[v] + 1$ với mọi cung dư (u, v) . Hàm độ cao trên là thích ứng với tiền luồng f .

```
1 void Init()
2 {
3     for (∀u ∈ V) excess(u) = 0;
4     for (∀e ∈ E) f(e) = 0;
5     for (e = (s, u) ∈ s → V) //Xét mọi cung e ra khỏi s
6         IncFlow(e, c(e)); //Đẩy lượng tiền luồng tối đa qua cung, làm cung bão hòa
7     for (∀u ∈ V) h[u] = 0; //Các đỉnh có độ cao 0
8     h[s] = n; //Ngoại trừ s có độ cao n
9 }
```

1.4.6. Thuật toán FIFO Preflow-Push

Ý tưởng của thuật toán đẩy tiền luồng khá đơn giản: Khởi tạo tiền luồng f và hàm độ cao, sau đó nếu thấy phép đẩy hay nâng nào thực hiện được thì thực hiện ngay... Cho tới khi không còn phép đẩy hay nâng nào có thể thực hiện được nữa thì tiền luồng f sẽ trở thành một luồng, đó là luồng cực đại trên mạng.

Thứ tự các phép toán cơ bản (đẩy và nâng) không ảnh hưởng tới giá trị luồng cực đại tìm được. Đã có nhiều nghiên cứu về cách thức chọn thực hiện phép toán cơ bản để để cải thiện độ phức tạp tính toán, qua đó đã đề xuất nhiều thuật toán đẩy tiền luồng khác nhau, chẳng hạn như FIFO Preflow-Push, Relabel-to-Front, Highest Label Preflow, v.v... Trong mục này ta sẽ trình bày một trong những thuật toán đó: FIFO Preflow-Push [4].

• Thứ tự xử lý các đỉnh tràn

Thuật toán trước tiên khởi tạo một tiền luồng và hàm độ cao tương ứng. Sau đó thuật toán xếp các đỉnh tràn vào trong một hàng đợi, từng đỉnh tràn u sẽ lấy ra khỏi hàng đợi và xử lý theo cách sau:

- ☀ Pha 1: Duyệt các cung ra khỏi u và thực hiện ngay phép $\text{Push}(\cdot)$ trên cung đó nếu được. Những đỉnh tràn mới tạo ra sau các phép $\text{Push}(\cdot)$ sẽ được xếp vào hàng đợi (nếu chúng chưa có trong hàng đợi). Phép duyệt này sẽ dừng ngay khi u hết tràn hoặc khi đã duyệt hết các cung ra khỏi u .
- ☀ Pha 2: Khi đã hết pha 1 mà u vẫn tràn, ta nâng u lên bằng phép $\text{Lift}(u)$ rồi xếp u vào hàng đợi chờ xử lý sau. Chú ý rằng sau pha 1 mà u vẫn tràn tức là không còn cung dư nối từ u xuống đỉnh thấp hơn. Đây chính là điều kiện để thực hiện phép $\text{Lift}(u)$.

Thuật toán kết thúc khi hàng đợi các đỉnh tràn đã rỗng, khi đó tiền luồng trên mạng đã trở thành luồng cực đại.

• Con trở tới cung còn hiệu lực

Trong thuật toán FIFO Preflow-Push, một đỉnh có thể bị đẩy vào và rút ra khỏi hàng đợi để xử lý nhiều lần. Mỗi khi một đỉnh tràn được lấy ra xử lý, ta cần quét tất cả các cung ra khỏi nó để thử thực hiện phép đẩy. Điều đó làm những cung bão hòa, những cung nối tới đỉnh cao hơn sẽ bị xét lại nhiều lần dù không thể thực hiện phép đẩy trên những cung đó. Tương tự như thuật toán Dinic, thuật toán FIFO Preflow-Push sử dụng các con trở tới cung còn hiệu lực để sớm bỏ qua những cung chắc chắn không thể đẩy luồng được.

Giả sử rằng có một đỉnh tràn u và một cung $e = (u, v)$ không thể thực hiện được phép đẩy, tức là ít nhất một trong hai điều kiện sau đây được thỏa mãn:

- ☀ (u, v) là cung bão hòa: $c_f(e) = 0$.
- ☀ u không cao hơn v : $h[u] \leq h[v]$.

Câu hỏi đặt ra là lúc nào phép đẩy có cơ hội thực hiện trên cung e ?

- ☀ Sau bất kỳ phép đẩy nào, chúng ta vẫn không thể thực hiện $\text{Push}(e)$. Thật vậy, nếu u không cao hơn v , thì sau bao nhiêu phép đẩy, u vẫn không cao hơn v . Nếu u cao hơn v thì e phải là cung bão hòa, phép đẩy duy nhất có thể biến e thành cung dư là $\text{Push}(-e)$ làm tăng $c_f(e)$. Nhưng phép $\text{Push}(-e)$ không thể thực hiện được vì cung $-e = (v, u)$ đang có v thấp hơn u .
- ☀ Sau bất kỳ phép nâng nào ngoại trừ $\text{Lift}(u)$, chúng ta cũng không thể thực hiện phép đẩy trên cung e . Bởi phép nâng không làm thay đổi dư lượng, nếu e đang bão hòa thì sau phép nâng nó vẫn bão hòa. Nếu e là cung dư thì u đang không cao hơn v , phép nâng duy nhất có thể khiến u cao hơn v là $\text{Lift}(u)$.

Nhận xét trên cho phép bỏ qua các cung chắc chắn không thực hiện được phép đẩy: Mỗi đỉnh u có danh sách $a[u]$ chứa các cung ra khỏi u . Sau khi khởi tạo, con trỏ $\text{ptr}[u]$ được đặt vào đầu danh sách $a[u]$. Con trỏ $\text{ptr}[u]$ mang ý nghĩa: Các cung đứng trước vị trí $\text{ptr}[u]$ chắc chắn không thể thực hiện phép đẩy.

Với mỗi đỉnh tràn u lấy khỏi hàng đợi, ta duyệt từ vị trí $\text{ptr}[u]$ để tìm cung thực hiện phép đẩy. Cho dù phép đẩy được thực hiện hay không, nếu sau đó mà u vẫn tràn thì cung tại vị trí $\text{ptr}[u]$ không thể thực hiện phép đẩy nữa, con trỏ $\text{ptr}[u]$ được dịch đi một vị trí sang cung kế tiếp... Quá trình lặp lại cho đến khi:

- ☀ Hoặc u hết tràn, thuật toán tiếp tục với đỉnh kế tiếp lấy ra từ hàng đợi...
- ☀ Hoặc con trỏ $ptr[u]$ đã duyệt hết danh sách $a[u]$ mà u vẫn tràn, khi đó phép $Lift(u)$ được thực hiện, con trỏ $ptr[u]$ đặt lại vào đầu danh sách $a[u]$, đỉnh u được xếp vào hàng đợi chờ xử lý sau...

• Sơ đồ cài đặt

Đến đây ta có thể viết một đoạn mã giả cho thuật toán FIFO Preflow-Push:

```

1 Init(); //Khởi tạo tiền luồng, hàm độ cao, tính lượng tràn tại các đỉnh
2 for ( $\forall u \in V$ )  $ptr[u] = a[u].begin()$ ; //Con trỏ  $ptr[u]$  đặt vào đầu danh sách  $a[u]$ 
3 overQueue = «Hàng đợi chứa các đỉnh tràn»;
4 while (overQueue  $\neq \emptyset$ )
5 {
6      $u = \text{«Đỉnh tràn lấy ra từ đầu hàng đợi overQueue»}$ ;
7     for (;  $ptr[u] \neq a[u].end()$ ;  $++ptr[u]$ ) //Duyệt các cung  $e \in a[u]$  từ con trỏ  $ptr[u]$ 
8     {
9          $e = (u, v) = \text{«Cung tại vị trí } ptr[u]\text{»}$ ;
10        if ( $cf(e) == 0 \parallel h[u] \leq h[v]$ ) //Phép đẩy không thực hiện được trên cung  $e$ 
11            continue; //Bỏ qua
12        if ( $v \neq s \ \&\& \ v \neq t \ \&\& \ excess[v] == 0$ ) //Sau khi Push(e) thì
13            «Xếp  $v$  vào cuối hàng đợi OverQueue»; //v trở thành đỉnh tràn mới
14        Push(e);
15        if ( $excess[u] == 0$ ) //u hết tràn sau phép đẩy
16            break; //quá trình xử lý đỉnh  $u$  kết thúc
17    }
18    if ( $excess[u] > 0$ ) //Sau khi xét hết  $a[u]$  mà  $u$  vẫn tràn.
19    {
20        Lift(u); //Nâng  $u$ 
21         $ptr[u] = a[u].begin()$ ; //ptr[u] đặt lại vào đầu danh sách  $a[u]$ 
22        «Xếp  $u$  vào cuối hàng đợi overQueue»;
23    }
24 }
```

1.4.7. Tính đúng của thuật toán

Tính đúng đắn của các thuật toán đẩy tiền luồng (bao gồm cả thuật toán FIFO Preflow-Push) được chỉ ra qua các nhận xét sau:

Bổ đề 1-14

Phép đẩy và phép nâng không làm tiền luồng vi phạm ràng buộc độ cao

Chứng minh

Giả sử f là một tiền luồng trên mạng $G = (V, E, c, s, t)$, h là một hàm độ cao ứng với tiền luồng f . Ta chứng minh rằng nếu thỏa mãn điều kiện thực hiện, phép đẩy và phép nâng vẫn đảm bảo h là hàm độ cao ứng với f .

Dĩ nhiên $h[s]$ luôn bằng n và $h[t]$ luôn bằng 0 vì chúng không phải đỉnh tràn, phép đẩy và nâng không thay đổi $h[s]$ và $h[t]$. Ta chỉ cần chứng minh với mọi cung dư $e = (u, v)$ thì $h[u] \leq h[v] + 1$.

Phép đẩy không làm thay đổi độ cao, ràng buộc độ cao vẫn thỏa mãn với mọi cung dư trước phép đẩy. Phép đẩy trên cung $e = (u, v)$ chỉ tăng dư lượng duy nhất một cung $-e = (v, u)$ và có thể làm nó đang bão hòa trở thành cung dư. Tuy nhiên điều kiện $h[u] > h[v]$ của phép đẩy cũng chỉ ra rằng $h[v] \leq h[u] + 1$. Ràng buộc độ cao thỏa mãn trên cung $-e$.

Phép nâng thì lại không làm phát sinh cung dư mới. Xét một phép nâng $\text{Lift}(u)$, vì chỉ có $h[u]$ tăng lên, ta chỉ cần chứng minh rằng buộc độ cao vẫn thỏa mãn trên các cung dư đi vào hoặc đi ra khỏi u :

- ☀ Với một cung dư (v, u) thì ràng buộc độ cao $h[v] \leq h[u] + 1$ vẫn thỏa mãn khi tăng $h[u]$.
- ☀ Với một cung dư (u, v) thì việc đặt: $h[u] = \min\{h[v] : (u, v) \in E_f\} + 1$ cũng suy ra $h[u] \leq h[v] + 1$ ■

Bổ đề 1-15

Nếu trên mạng còn đỉnh tràn, chắc chắn có thể thực hiện được phép đẩy hoặc phép nâng

Chứng minh

Xét một đỉnh tràn u , nếu không thực hiện được phép đẩy thì sẽ không có cung dư nối từ u tới đỉnh thấp hơn. Đây chính là điều kiện để thực hiện phép nâng đỉnh u ■

Bổ đề 1-16

Cho f là một tiền luồng trên mạng $G = (V, E, c, s, t)$ và h là một hàm độ cao ứng với f . Khi đó với mọi đường dư, hiệu số:

$$\text{Độ cao đỉnh đầu} - \text{Độ cao đỉnh cuối}$$

không vượt quá độ dài đường đi.

Tức là với mọi đường dư $P = \langle v_0, v_1, \dots, v_k \rangle$ ta có:

$$h[v_0] - h[v_k] \leq k$$

Chứng minh

Từ ràng buộc độ cao:

$$h[u] \leq h[v] + 1, \forall (u, v) \in E_f$$

nên nếu đi từ đầu tới cuối đường P , mỗi khi qua một cung thì độ cao sẽ tăng lên, giữ nguyên, hoặc giảm đi chỉ 1 đơn vị. Điều đó chỉ ra rằng nếu đường P chứa k cung dư thì khi tới điểm cuối v_k , độ cao cùng lắm là giảm đi k đơn vị so với điểm đầu v_0 , tức là

$$h[v_0] - h[v_k] \leq k \blacksquare$$

Hệ quả

Khi thuật toán đẩy tiền luồng kết thúc, sẽ không còn tồn tại đường tăng luồng.

Chứng minh

Khi thuật toán kết thúc, thuật toán còn trả về một hàm độ cao ứng với tiền luồng thu được. Giả sử có đường tăng luồng trên mạng dư lượng. Vì đường tăng luồng phải là đường đi đơn $s \rightsquigarrow t$, độ dài đường tăng luồng không thể vượt quá $n - 1$. theo Bổ đề 1-16 ta có:

$$h[s] - h[t] \leq n - 1$$

Tuy nhiên, $h[t] = 0$, $h[s] = n$ sự mâu thuẫn này cho thấy không thể tồn tại đường tăng luồng trên G_f .

Định lý 1-17

Thuật toán đẩy tiền luồng trả về luồng cực đại trên mạng

Chứng minh

Khi không còn có thể thực hiện bất kỳ phép đẩy hay phép nâng nào, thuật toán kết thúc và trả về một tiền luồng f . Bổ đề 1-15 cho biết không còn đỉnh tràn trên mạng. Tức là $\forall u \notin \{s, t\}$ ta có:

$$Excess[u] = f(u \rightarrow V) - f(V \rightarrow u) = 0$$

Tiền luồng f thỏa mãn ràng buộc về tính bảo toàn và vì vậy nó trở thành một luồng.

Ngoài ra ta còn thu được một hàm độ cao ứng với luồng f , Bổ đề 1-16 khẳng định không còn đường tăng luồng và Định lý 1-9 đã chứng minh khi không tồn tại đường tăng luồng thì f là luồng cực đại ■

1.4.8. Tính dừng của thuật toán

Tính dừng của thuật toán được chỉ ra khi phân tích thời gian thực hiện giải thuật. Ta sẽ chỉ phân tích thời gian thực hiện giải thuật FIFO Preflow-Push.

Bổ đề 1-18

Cho f là một tiền luồng trên mạng $G = (V, E, c, s, t)$, khi đó với mọi đỉnh tràn u , tồn tại một đường dư đi từ u tới s .

Chứng minh

Với một đỉnh tràn u bất kỳ, xét tập X là tập các đỉnh có thể đến được từ u bằng một đường dư và $Y = V \setminus X$.

Xét tổng mức tràn của các đỉnh $\in X$:

$$\begin{aligned} \sum_{u \in X} excess(u) &= \sum_{u \in X} (f(V \rightarrow u) - f(u \rightarrow V)) \\ &= \sum_{u \in X} f(V \rightarrow u) - \sum_{u \in X} f(u \rightarrow V) \\ &= f(V \rightarrow X) - f(X \rightarrow V) \\ &= f(Y \rightarrow X) + f(X \rightarrow X) - f(X \rightarrow Y) - f(X \rightarrow X) \\ &= f(Y \rightarrow X) - f(X \rightarrow Y) \end{aligned}$$

Theo cách xây dựng tập X và Y , sẽ không có cung dư $\in X \rightarrow Y$, tức là tiền luồng trên mọi cung $\in Y \rightarrow X$ phải bằng 0. Vậy:

$$\begin{aligned} \sum_{u \in X} excess(u) &= \underbrace{f(Y \rightarrow X)}_{=0} - \underbrace{f(X \rightarrow Y)}_{\geq 0} \\ &\leq 0 \end{aligned}$$

Lượng tràn tại mỗi đỉnh không phải đỉnh phát đều là số không âm, ngoài ra u là đỉnh tràn $\in X$ nên $excess(u) > 0$, điều này cho thấy chắc chắn đỉnh phát s phải thuộc X để $excess(s) \leq 0$. Nói cách khác từ u đến được s bằng một đường dư ■

Hệ quả

Trong thuật toán đẩy tiền luồng, độ cao của các đỉnh nhỏ hơn $2n$.

Chứng minh

Phép khởi tạo độ cao:

$$h[u] = \begin{cases} n, & \text{nếu } u = s \\ 0, & \text{nếu } u \neq s \end{cases}$$

cho độ cao các đỉnh nhỏ hơn $2n$.

Độ cao của s và t không bao giờ bị thay đổi. Với mỗi đỉnh $u \notin \{s, t\}$ thì chỉ phép $\text{Lift}(u)$ có thể làm tăng độ cao của đỉnh u . Nếu có phép $\text{Lift}(u)$ được thực hiện thì u phải là đỉnh tràn trước và sau khi thực hiện $\text{Lift}(u)$. Bổ đề 1-18 cho biết tồn tại đường đi dư $u \rightsquigarrow s$ và bởi đường dư ngắn nhất $u \rightsquigarrow s$ có độ dài $< n$, theo Bổ đề 1-16 ta có:

$$h[u] - h[s] < n$$

Từ $h[s] = n$, ta có $h[u] < 2n$ ■

Định lý 1-19 (Thời gian thực hiện giải thuật FIFO Preflow-Push)

Thuật toán FIFO Preflow-Push tìm được luồng cực đại trong thời gian $O(n^3 + n * m)$

Chứng minh

(a) Số phép nâng là $O(n^2)$ và tổng thời gian thực hiện chúng là $O(n * m)$

Mỗi phép nâng tăng độ cao một đỉnh lên ít nhất 1 đơn vị, độ cao của mỗi đỉnh luôn nhỏ hơn $2n$ (hệ quả của Bổ đề 1-18). Vậy nên tổng số phép nâng trong thuật toán là $O(n^2)$. Mỗi cung $e \in u \rightarrow V$ sẽ được xét đến đúng một lần trong phép $\text{Lift}(u)$, phép $\text{Lift}(u)$ lại thực hiện $< 2n$ lần. Vậy tổng cộng trong tất cả các phép nâng thì mỗi cung sẽ được xét ít hơn $2n$ lần, mạng có m cung nên tổng thời gian thực hiện của các phép nâng là $O(n * m)$.

(b) Số phép đẩy bão hòa cũng như tổng thời gian thực hiện chúng là $O(n * m)$

Xét một cung $e = (u, v)$. Sau phép đẩy bão hòa $\text{Push}(e)$ thì v thấp hơn u . Muốn thực hiện $\text{Push}(e)$ một lần nữa thì trước đó chắc chắn phải có phép đẩy $\text{Push}(-e)$ để biến e trở lại thành cung dư. Tuy nhiên để thực hiện $\text{Push}(-e)$ thì v đã phải được nâng lên cao hơn u . Nói cách khác, giữa hai phép đẩy bão hòa $\text{Push}(e)$ thì độ cao đỉnh v tăng lên ít nhất 2 đơn vị. Độ cao đỉnh luôn nhỏ hơn $2n$ nên số phép đẩy bão hòa trên một cung là $O(n)$. Mạng có m cung nên số phép đẩy bão hòa cũng như thời gian thực hiện chúng trong thuật toán là $O(n * m)$.

(c) Số phép đẩy không bão hòa cũng như tổng thời gian thực hiện chúng là $O(n^3)$

Chia dãy các thao tác đẩy và nâng làm các pha liên tiếp. Pha thứ nhất bắt đầu khi hàng đợi được khởi tạo gồm các đỉnh tràn và kết thúc khi tất cả các đỉnh đó (và chỉ những đỉnh đó thôi) đã được lấy ra khỏi hàng đợi và xử lý. Pha thứ hai tiếp tục với hàng đợi gồm những đỉnh được đẩy vào trong pha thứ nhất và kết thúc khi tất cả các đỉnh này được lấy ra khỏi hàng đợi và xử lý, pha thứ ba, thứ tư... được chia ra theo cách tương tự như vậy.

Định nghĩa hàm thế (*potential function*) Φ là độ cao lớn nhất của các đỉnh tràn:

$$\Phi = \max\{h[v] : v \text{ là đỉnh tràn}\}$$

Trong trường hợp mạng không còn đỉnh tràn thì ta quy ước $\Phi = 0$. Vậy $\Phi = 0$ sau khi khởi tạo tiền luồng và trở lại bằng 0 khi thuật toán kết thúc.

Phép đẩy $\text{Push}(e = (u, v))$ chỉ thực hiện trên cung đi từ đỉnh cao xuống đỉnh thấp, Nếu v được xếp vào hàng đợi thì nó luôn thấp hơn đỉnh u vừa lấy ra khỏi hàng đợi. Suy ra nếu một pha không chứa phép nâng thì giá trị hàm thế Φ sau pha đó giảm đi ít nhất 1 đơn vị.

Giá trị hàm thế Φ chỉ có thể tăng lên hoặc giữ nguyên sau một pha nếu pha đó có chứa phép nâng. Khi ấy giá trị Φ mới phải bằng một độ cao của một đỉnh tràn v nào đó, tức là mức tăng của Φ không vượt quá mức tăng độ cao của v :

$$\begin{aligned}\Phi_{\text{mới}} - \Phi_{\text{cũ}} &= h[v]_{\text{mới}} - \Phi_{\text{cũ}} \\ &\leq h[v]_{\text{mới}} - h[v]_{\text{cũ}}\end{aligned}$$

Độ cao của mỗi đỉnh được khởi tạo bằng 0 và được nâng lên tối đa bằng $2n - 1$ nên tổng toàn bộ mức tăng của các đỉnh là $O(n^2)$. Suy ra tổng mức tăng của Φ trên các pha có chứa phép nâng là $O(n^2)$. Vì Φ cuối cùng phải bằng 0, số các pha làm Φ giảm cũng phải là $O(n^2)$. Nói cách khác, sẽ chỉ có $O(n^2)$ pha không chứa phép nâng.

Ngoài ra ta biết rằng số phép nâng là $O(n^2)$ nên số pha có chứa phép nâng cũng là $O(n^2)$. Cộng lại ta có số pha cần thực hiện trong toàn bộ giải thuật là $O(n^2)$.

Một pha sẽ phải lấy khỏi hàng đợi tối đa $n - 2$ đỉnh tràn để xử lý. Với mỗi đỉnh lấy từ hàng đợi, tối đa 1 phép đẩy không bão hòa được thực hiện vì sau phép đẩy này thì đỉnh sẽ hết tràn. Vậy trong mỗi pha có không quá $n - 2$ phép đẩy không bão hòa. Vì có $O(n^2)$ pha, số phép đẩy không bão hòa trong cả giải thuật là $O(n^3)$. Tổng thời gian thực hiện các phép đẩy không bão hòa cũng là $O(n^3)$.

(d) Tổng thời gian duyệt danh sách liên thuộc bằng các con trỏ $ptr[.]$ là $O(n * m)$

Ban đầu mỗi con trỏ $ptr[u]$ được đặt vào đầu danh sách $a[u]$. Mỗi khi đỉnh tràn u được lấy khỏi hàng đợi, danh sách $a[u]$ được duyệt từ vị trí con trỏ $ptr[u]$ về cuối danh sách. Khi duyệt hết danh sách $a[u]$ mà u vẫn tràn thì sẽ có một phép $Lift(u)$ được thực hiện và con trỏ $ptr[u]$ được đặt lại về đầu danh sách $a[u]$. Số phép $Lift(u)$ nhỏ hơn $2n$, nên số lượt dịch con trỏ $ptr[u]$ nhỏ hơn $2 * n * |a[u]|$. Suy ra nếu xét tổng thể, số phép dịch các con trỏ $ptr[.]$ trên tất cả các danh sách liên thuộc phải nhỏ hơn:

$$2 * n * \sum_{u \in V} |a[u]| = 2 * n * m$$

Kết luận:

- ☀ Tổng thời gian thực hiện các phép nâng: $O(n * m)$.
- ☀ Tổng thời gian thực hiện các phép đẩy bão hòa: $O(n * m)$.
- ☀ Tổng thời gian thực hiện các phép đẩy không bão hòa: $O(n^3)$.
- ☀ Tổng thời gian thực hiện các phép duyệt danh sách liên thuộc bằng con trỏ $ptr[.]$: $O(n * m)$.

Thời gian thực hiện giải thuật FIFO Preflow-Push: $O(n^3 + n * m)$.

1.4.9. Mẹo giải tăng tốc độ chương trình

Người ta đã tìm ra được những bộ dữ liệu cụ thể cho trường hợp xấu của các thuật toán đẩy tiền luồng trong đó có thuật toán FIFO Preflow-Push, thậm chí còn chứng minh được rằng trong trường hợp xấu nhất, thuật toán FIFO Preflow-Push thực hiện trong thời gian $\Omega(n^3 + n * m)$.

Các thử nghiệm trên dữ liệu ngẫu nhiên cũng cho thấy rằng các thuật toán đẩy tiền luồng rất chậm nếu không sử dụng những mẹo giải (*heuristics*). Dù chưa có đánh giá lý thuyết nào về tác động của những mẹo giải lên thời gian thực hiện giải thuật nhưng những khảo sát thực tế đều cho thấy mẹo giải là bắt buộc đối với các thuật toán đẩy tiền luồng.

• Gán nhãn lại toàn bộ

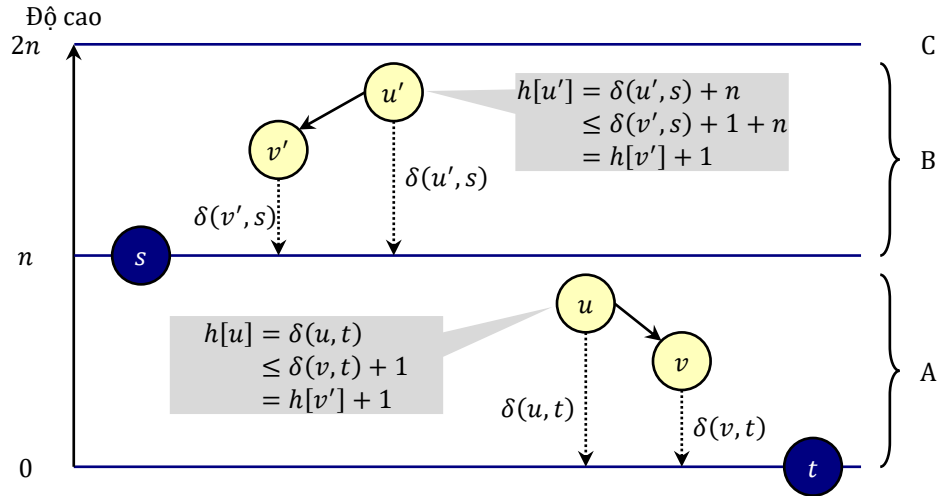
Với f là một tiền luồng trên mạng $G = (V, E, c, s, t)$. Phép gán nhãn lại toàn bộ (*global relabeling heuristic*) xét ba tập rời nhau:

- ☀ A : Tập các đỉnh có đường dư tới t .
- ☀ B : Tập các đỉnh $\notin T$ có đường dư tới s .
- ☀ C : Tập các đỉnh còn lại

Với hai đỉnh $u, v \in V$, gọi $\delta(u, v)$ là độ dài đường dư ngắn nhất từ u tới v . Độ cao của các đỉnh được đặt lại toàn bộ theo công thức:

$$h[u] = \begin{cases} \delta(u, t) & , \text{ nếu } u \in A \\ \delta(u, s) + n, & \text{ nếu } u \in B \\ 2n & , \text{ nếu } u \in C \end{cases}$$

Phép gán nhãn lại toàn bộ làm độ cao của các đỉnh không giảm đi và vẫn thích ứng với tiền luồng hiện có. Điều này dễ dàng chứng minh từ cách xây dựng ba tập A, B, C và bất đẳng thức tam giác của độ dài đường đi ngắn nhất (Hình 1-7).



Hình 1-7. Phép gán nhãn lại toàn bộ

Hệ quả của Bổ đề 1-18 đã chứng minh rằng:

$$\forall u \in C: h[u] < 2n$$

Tức là độ cao của các đỉnh không bị giảm đi sau phép gán nhãn lại toàn bộ. Việc đặt độ cao của các đỉnh $\in C$ thành $2n$ tuy có mâu thuẫn với khẳng định của Bổ đề 1-18 (độ cao của các đỉnh nhỏ hơn $2n$) nhưng không ảnh hưởng tới tính đúng của thuật toán vì những đỉnh $\in C$ sẽ không bao giờ tràn và có thể loại bỏ.

Việc tính lại độ cao của các đỉnh được thực hiện bằng hai lượt BFS từ t và s . Phép gán nhãn lại toàn bộ cũng có thể sử dụng để khởi tạo độ cao, và được thực hiện sau mỗi $O(n)$ phép nâng để không làm ảnh hưởng tới đánh giá $O(\cdot)$ của thời gian thực hiện giải thuật. Chú ý là sau phép gán nhãn lại toàn bộ, tất cả con trỏ $ptr[u]$ phải đặt lại vào đầu danh sách $a[u]$.

Có một quan sát thực tế rằng việc tháo luồng nên thực hiện từ đỉnh cao nhất tới đỉnh thấp nhất sẽ có lợi hơn (thay thế hàng đợi các đỉnh tràn bằng hàng đợi ưu tiên). Tuy vậy, các thử nghiệm trên dữ liệu ngẫu nhiên cho thấy cách làm này không cải thiện thời gian thực

thì, lại phải thêm nhiều kỹ thuật để không làm tăng độ phức tạp trong đánh giá $O(\dots)$. Để tránh một vài trường hợp dữ liệu xấu, ta sẽ lợi dụng BFS để sắp xếp lại hàng đợi các đỉnh tràn từ cao xuống thấp sau mỗi phép gán nhãn lại toàn bộ: Hàng đợi các đỉnh tràn là hàng đợi hai đầu, được làm rỗng trước phép gán nhãn lại toàn bộ; Khi thuật toán BFS thăm một đỉnh tràn, đỉnh đó sẽ được đặt vào đầu hàng đợi các đỉnh tràn, đảm bảo các đỉnh tràn được xếp từ cao xuống thấp từ đầu đến cuối hàng đợi.

Để ý rằng nếu không cài đặt phép nâng, thay vào đó thuật toán gán nhãn lại toàn bộ khi không thực hiện được phép đẩy. Khi ấy thuật toán FIFO Preflow-Push hoạt động rất giống với thuật toán Dinic.

• Đẩy nhãn theo khe

Phép đẩy nhãn theo khe (*gap heuristic*) được thực hiện nhờ quan sát sau:

Giả sử ta có một số nguyên *gap* mà không đỉnh nào có độ cao *gap* (số nguyên *gap* này được gọi là “khe”), khi đó từ một đỉnh cao hơn *gap* sẽ không có cung dư tới đỉnh thấp hơn *gap*.

Phép đẩy nhãn theo khe nếu phát hiện khe $0 < gap < n$ sẽ xét tất cả những đỉnh $z \in V - \{s\}$ có $gap < h(z) \leq n$ và đặt lại $h[z] = n + 1$.

Mặc dù mẹo đẩy nhãn theo khe này rất tốt trên các bộ dữ liệu ngẫu nhiên, vẫn có những trường hợp xấu khiến cho mẹo giải này trở nên không hiệu quả. Vì vậy ta chỉ nên sử dụng nó khi kết hợp cùng với phép gán nhãn lại toàn bộ. Khi cần quan tâm tới tính đơn giản của chương trình, phép gán nhãn lại toàn bộ có thể coi là đủ tốt cho thuật toán FIFO Preflow-push nên không cần cài đặt thêm phép đẩy nhãn theo khe nữa.

1.4.10. Cài đặt

Thuật toán FIFO Preflow-Push sẽ được cài đặt với khuôn dạng Input/Output như trong chương trình cài đặt thuật toán Edmonds-Karp và thuật toán Dinic.

Việc tổ chức dữ liệu gần như kế thừa hoàn toàn từ thuật toán Dinic:

- ☀ Mỗi cung chứa thông tin về hai đỉnh đầu cuối: x, y và dư lượng c_f
- ☀ m cung thật và m cung giả được lưu trữ trong danh sách $e[0 \dots 2m)$, cung thật ở vị trí chẵn, cung giả ở vị trí lẻ, cung đối của $e[i]$ là cung $e[i \wedge 1]$
- ☀ Mỗi đỉnh $u \in [0 \dots n - 1]$ có
 - ⚙ Danh sách $a[u]$ chứa chỉ số các cung đi ra khỏi u
 - ⚙ Con trỏ $ptr[u]$ trỏ tới cung còn hiệu lực trong danh sách $a[u]$
 - ⚙ Độ cao $h[u]$
 - ⚙ Lượng tràn $excess[u]$

Dưới đây là chương trình cài đặt thuật toán FIFO Preflow-Push kết hợp với kỹ thuật gán nhãn lại toàn bộ, việc cài đặt và đánh giá hiệu suất của phép đẩy nhãn theo khe coi như bài tập. Các bạn có thể thử cài đặt kết hợp cả hai kỹ thuật tăng tốc này để xác định xem việc đó có thực sự cần thiết không. Input/Output có khuôn dạng giống như ở chương trình cài đặt thuật toán Edmonds-Karp. Tương tự như trong cài đặt thuật toán Edmonds-Karp và thuật toán Dinic, ta sẽ chỉ duy trì dư lượng trên các cung.

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <algorithm>
5  using namespace std;
6  using lli = long long;
7  const int MAX_N = 1e4;
8  const int MAX_M = 1e5;
9
10 struct TEdge //Cấu trúc một cung
11 {
12     int x, y; //Hai đỉnh đầu nút
13     int cf;    //Dư lượng
14 };
15
16 int n, m, s, t;
17 TEdge e[2 * MAX_M];
18 vector<int> a[MAX_N]; //Danh sách liên thuộc
19 vector<int>::const_iterator ptr[MAX_N]; //ptr[u] là cung đang xét trong a[u]
20 int h[MAX_N]; //Độ cao các đỉnh
21 lli excess[MAX_N]; //Lượng tràn ở các đỉnh
22 deque<int> overQueue;
23
24 void Enter()
25 {
26     cin >> n >> m >> s >> t;
27     for (int i = 0; i < 2 * m; i += 2)
28     {
29         int u, v, cap;
30         cin >> u >> v >> cap; //Đọc một cung (u, v) và sức chứa cap
31         e[i] = {u, v, cap}; //Thêm cung (u, v) với dư lượng cap
32         e[i + 1] = {v, u, 0}; //Thêm cung đối (v, u) với dư lượng 0
33         if (u != v) //Không xét các khuyên u == v
34         {
35             a[u].push_back(i);
36             a[v].push_back(i + 1);
37         }
38     }
39 }
40
41 void IncFlow(int i, int delta) //Đẩy thêm delta đơn vị luồng theo cung e[i]
42 {
43     e[i].cf -= delta; //Dư lượng cung e[i] giảm delta
44     e[i ^ 1].cf += delta; //Dư lượng cung đối tăng delta
45     excess[e[i].x] -= delta; //Lượng tràn tại e[i].x giảm delta
46     excess[e[i].y] += delta; //Lượng tràn tại e[i].y tăng delta
47 }
48

```

```

49 void BFS(int Finish) //BFS để đo  $h[v]$  = độ dài đường dư ngắn nhất từ v tới Finish
50 {
51     queue<int> q({Finish}); //Hàng đợi dùng cho BFS ban đầu chỉ có Finish
52     while (!q.empty())
53     {
54         int u = q.front(); q.pop(); //Lấy u khỏi hàng đợi
55         if (u != s && u != t && excess[u] > 0) //u là đỉnh tràn
56             overQueue.push_front(u); //Xếp u vào đầu overQueue
57         for (int i: a[u]) //Xét các cung  $e[i] \in (u \rightarrow v)$ 
58         {
59             int v = e[i].y; //Cung đối  $e[i \wedge 1] = (v, u)$  đi vào u
60             if (e[i  $\wedge$  1].cf == 0 ||  $h[v] < 2 * n$ ) //cung đối bão hòa hoặc v đã thăm
61                 continue; //Bỏ qua
62              $h[v] = h[u] + 1$ ; //Đặt nhãn  $h[v]$  = độ dài ĐĐNN tới Finish
63             q.push(v); //Xếp v vào hàng đợi
64         }
65     }
66 }
67
68 void GlobalRelabel() //Thuật toán gán nhãn lại toàn bộ
69 {
70     fill(h, h + n, 2 * n); //h[.] = 2 * n ứng với đỉnh chưa thăm
71      $h[t] = 0$ ;  $h[s] = n$ ;
72     overQueue.clear(); //Xóa hàng đợi các đỉnh tràn để xếp lại từ cao tới thấp
73     BFS(t); //BFS từ t:  $h[v]$  = độ dài đường dư ngắn nhất từ v tới t
74     BFS(s); //BFS từ s:  $h[v] = n +$  độ dài đường đi ngắn nhất từ v tới s
75     for (int u = 0; u < n; ++u) //Chỉnh lại các con trỏ ptr[.]
76         ptr[u] = a[u].begin(); //về đầu các danh sách a[.]
77 }
78
79 void Init() //Khởi tạo tiền luồng
80 {
81     fill(excess, excess + n, 0); //Mức tràn ban đầu đặt bằng 0
82     for (int i: a[s]) //Xét các cung  $e[i]$  ra khỏi s
83         IncFlow(i, e[i].cf); //Thêm tiền luồng tối đa làm  $e[i]$  bão hòa
84     GlobalRelabel(); //Khởi tạo độ cao, con trỏ ptr[.], hàng đợi các đỉnh tràn
85 }
86
87 void Push(int i) //Phép đẩy trên cung  $e[i]$ 
88 {
89     IncFlow(i, min(excess[e[i].x], (lli)e[i].cf));
90 }
91
92 void Lift(int u) //Phép nâng
93 {
94      $h[u] = 2 * n$ ;
95     for (int i: a[u]) //Khuyến đã được loại bỏ, chắc chắn  $e[i]$  không phải khuyến
96         if (e[i].cf > 0)  $h[u] = \min(h[u], h[e[i].y])$ ;
97     ++h[u];
98 }
99

```

```

100 void FIFOPreFlowPush() //Thuật toán FIFO Preflow-Push
101 {
102     int LiftCount = 0;
103     while (!overQueue.empty())
104     {
105         int u = overQueue.front(); overQueue.pop_front();
106         for (; ptr[u] != a[u].end(); ++ptr[u]) //Duyệt a[u] từ vị trí con trỏ ptr[u]
107         {
108             int i = *ptr[u]; //Xét cung e[i] = (u, v)
109             int v = e[i].y;
110             if (e[i].cf == 0 || h[u] <= h[v]) //Phép đẩy không thực hiện được trên e[i]
111                 continue; //bỏ qua, xét cung kế tiếp
112             if (v != s && v != t && excess[v] == 0) //v ∉ {s,t} và v ∉ Q thì
113                 overQueue.push_back(v); //sau phép đẩy v sẽ thành đỉnh tràn mới
114             Push(i); //Thực hiện phép đẩy trên cung e[i]
115             if (excess[u] == 0) //u hết tràn
116                 break; //dừng tháo luồng tại u
117         }
118         if (excess[u] > 0) //Sau khi xét hết a[u] mà u vẫn tràn.
119         {
120             Lift(u); //Nâng u
121             ptr[u] = a[u].begin(); //Đặt lại con trỏ ptr[u] vào cung đầu tiên ra khỏi u
122             overQueue.push_back(u); //Đẩy u vào hàng đợi chờ xử lý sau
123             if (++LiftCount % n == 0) //Đã qua n phép nâng
124                 GlobalRelabel(); //Gán nhãn lại toàn bộ, sắp xếp overQueue từ cao-thấp
125         }
126     }
127 }
128
129 void Print() //In kết quả
130 {
131     cout << "Value of flow: " << excess[t] << '\n'; //Giá trị luồng = lượng tràn tại t
132     //Chỉ ra sức chứa và luồng trên các cung đã cho bằng thông tin về dư lượng
133     for (int i = 0; i < 2 * m; i += 2) //Cung thật mang chỉ số chẵn trong e
134     {
135         cout << "e[" << i / 2 << "] = "
136             << "(" << e[i].x << ", " << e[i].y << "): "
137             << "c = " << e[i].cf + e[i ^ 1].cf << ", " //c(e) = cf(e) + cf(-e)
138             << "f = " << e[i ^ 1].cf << '\n'; //f(e) = cf(-e)
139     }
140
141 int main()
142 {
143     Enter(); //Nhập dữ liệu
144     Init();
145     FIFOPreFlowPush();
146     Print(); //In kết quả
147 }

```

Định lý 1-20 (Định lý về tính nguyên)

Nếu tất cả các sức chứa là số nguyên, đồng thời luồng/tiền luồng khởi tạo là các số nguyên, thì thuật toán Edmonds-Karp, thuật toán Dinic, cũng như thuật toán đẩy tiền luồng luôn tìm được luồng cực đại với luồng trên cung là các số nguyên.

Chứng minh

Đối với thuật toán Edmonds-Karp, ban đầu ta khởi tạo luồng nguyên. Mỗi lần tăng luồng dọc theo đường tăng luồng P , luồng trên mỗi cung hoặc giữ nguyên, hoặc tăng/giảm một lượng Δ_P cũng là số nguyên. Vậy nên cuối cùng luồng cực đại phải có giá trị nguyên trên tất cả các cung. Tương tự với thuật toán Dinic.

Đối với thuật toán đẩy tiền luồng, ban đầu ta khởi tạo một tiền luồng trên các cung là số nguyên. Phép đẩy và nâng không làm thay đổi tính nguyên của tiền luồng trên các cung. Vậy nên khi thuật toán kết thúc, tiền luồng trở thành luồng cực đại với giá trị luồng trên các cung là số nguyên.

1.5. So sánh về hiệu suất thực tế

Chính vì các kết quả lý thuyết về thời gian thực hiện giải thuật chỉ là đánh giá trong trường hợp xấu nhất, khá lệch so với thực tế gồm những dữ liệu cụ thể và ngẫu nhiên, ta sẽ thử nghiệm các thuật toán đã trình bày trong bài với các bộ dữ liệu ngẫu nhiên nhằm đưa ra nhận định nên dùng thuật toán nào trong trường hợp nào.

Dưới đây là bảng so sánh tốc độ của các chương trình cài đặt cụ thể trên một số bộ dữ liệu. Với một cặp số n, m , 100 đồ thị với n đỉnh, m cung được sinh ngẫu nhiên. Có 4 chương trình được thử nghiệm: A: Thuật toán Edmonds-Karp, B: thuật toán Dinic, C: thuật toán FIFO Preflow-Push và D: Thuật toán FIFO Preflow-Push với phép gán nhãn lại toàn bộ. Mỗi chương trình được thử trên cả 100 đồ thị và đo thời gian thực hiện trung bình (tính bằng giây):

Số đỉnh (n)	Số cung (m)	A	B	C	D
100	10000	0.0705	0.0221	0.0331	0.0226
200	30000	0.5004	0.0541	0.1282	0.0556
500	40000	1.0985	0.0709	0.2770	0.0722
1000	10000	0.1551	0.0237	0.1343	0.0239
1000	100000	6.5447	0.1704	1.3981	0.1729
10000	100000	19.0872	0.1857	13.6925	0.1864

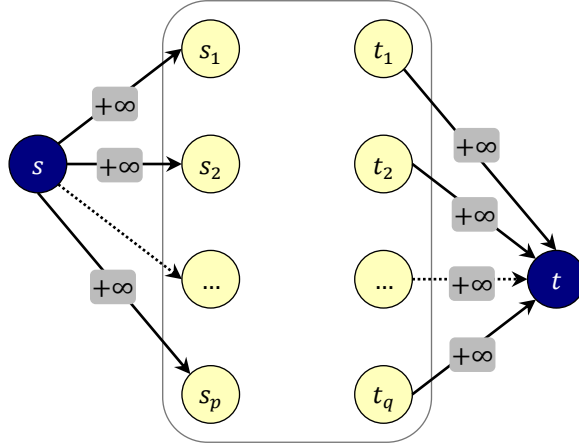
Những con số về thời gian thực thi chương trình cho thấy thuật toán A và C không thích hợp khi xử lý với mạng kích thước lớn. Thuật toán (B) và (D) có tốc độ tốt trong trường hợp mạng ngẫu nhiên, tuy nhiên phải nhấn mạnh rằng tuy thuật toán B (Dinic) có tốc độ tốt nhất trong các mạng ngẫu nhiên, có những cấu hình mạng khiến nó rơi vào trường hợp xấu như ta đã chỉ ra trong bài. Vì vậy khi triển khai trên các mạng kích thước lớn, sự lựa chọn an toàn vẫn là thuật toán FIFO Preflow-push với mẹo giải gán nhãn lại toàn bộ sau mỗi $|V|$ phép $Lift()$.

1.6. Một số mở rộng và ứng dụng của luồng

1.6.1. Mạng với nhiều đỉnh phát và nhiều đỉnh thu

Ta mở rộng khái niệm mạng bằng cách cho phép mạng G có p đỉnh phát: $s_1, s_2, \dots, s_p$ và q đỉnh thu $t_1, t_2, \dots, t_q$, các đỉnh phát và các đỉnh thu hoàn toàn phân biệt. Hàm sức chứa và luồng trên mạng được định nghĩa tương tự như trong trường hợp mạng có một đỉnh phát và một đỉnh thu. Giá trị của luồng được định nghĩa bằng tổng luồng trên các cung đi ra khỏi các đỉnh phát. Bài toán đặt ra là tìm luồng cực đại trên mạng có nhiều đỉnh phát và nhiều đỉnh thu.

Thêm vào mạng hai đỉnh: một siêu đỉnh phát s và siêu đỉnh thu t . Thêm các cung nối từ s tới các đỉnh s_i có sức chứa $+\infty$, thêm các cung nối từ các đỉnh t_j tới t với sức chứa $+\infty$. Ta được một mạng mới $G' = (V, E')$ (Hình 1-8).



Hình 1-8. Mạng với nhiều đỉnh phát và nhiều đỉnh thu

Có thể thấy rằng nếu f là một luồng cực đại trên G' , thì f hạn chế trên G cũng là luồng cực đại trên G . Vậy để tìm luồng cực đại trên G , ta sẽ tìm luồng cực đại trên G' rồi loại bỏ siêu đỉnh phát s , siêu đỉnh thu t và tất cả những cung giả mới thêm vào.

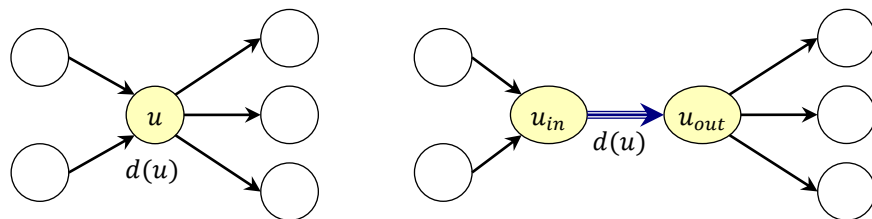
Một cách khác có thể thực hiện để tìm luồng trên mạng có nhiều đỉnh phát và nhiều đỉnh thu là loại bỏ tất cả các cung đi vào các đỉnh phát cũng như các cung đi ra khỏi các đỉnh thu. Chập tất cả các đỉnh phát thành một siêu đỉnh s và chập tất cả các đỉnh thu lại thành một siêu đỉnh thu t , mạng không còn các đỉnh $s_1, s_2, \dots, s_p$ và $t_1, t_2, \dots, t_q$ nữa mà chỉ có thêm đỉnh phát s và đỉnh thu t . Trên mạng ban đầu, mỗi cung đi vào/ra s_i được chỉnh lại đầu nút để nó đi vào/ra đỉnh s , mỗi cung đi vào/ra t_j cũng được chỉnh lại đầu nút để nó đi vào/ra đỉnh t , ta được một mạng mới G'' .

Khi đó ta có thể tìm f là một luồng cực đại trên G'' và khôi phục lại đầu nút của các cung như cũ để f trở thành luồng cực đại trên G .

1.6.2. Mạng với sức chứa trên cả các đỉnh và các cung

Cho mạng $G = (V, E, c, s, t)$, mỗi đỉnh $u \in V - \{s, t\}$ được gán một số không âm $d(u)$ gọi là sức chứa của đỉnh đó. Luồng φ trên mạng này được định nghĩa với tất cả các ràng buộc của luồng và thêm một điều kiện: Tổng luồng trên các cung đi vào/đi ra mỗi đỉnh $u \in V - \{s, t\}$ không được vượt quá $d(u)$: $\varphi(V, u) = \varphi(u, V) \leq d(u)$. Bài toán đặt ra là tìm luồng cực dương đại trên mạng có ràng buộc sức chứa trên cả các đỉnh và các cung.

Tách mỗi đỉnh $u \in V - \{s, t\}$ thành 2 đỉnh mới u_{in}, u_{out} và một cung (u_{in}, u_{out}) với sức chứa $d(u)$. Các cung đi vào u được chỉnh lại đầu nút để đi vào u_{in} và các cung đi ra khỏi u được chỉnh lại đầu nút để đi ra khỏi u_{out} (Hình 1-9). Ta xây dựng được mạng $G' = (V', E')$ với đỉnh phát s và đỉnh thu t .



Hình 1-9. Tách đỉnh

Khi đó việc tìm luồng cực đại trên mạng G có thể thực hiện bằng cách tìm luồng cực đại trên mạng G' , sau đó chập tất cả các cặp (u_{in}, u_{out}) trở lại thành đỉnh u để khôi phục lại mạng G ban đầu.

1.6.3. Mạng với ràng buộc luồng bị chặn hai phía

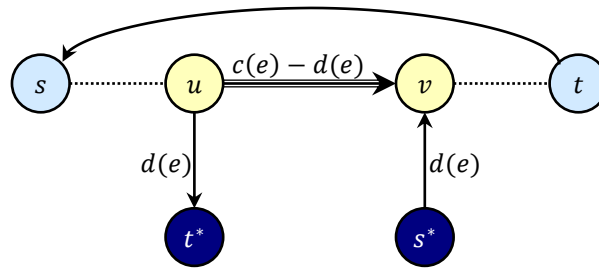
Cho mạng $G = (V, E, c, s, t)$ trong đó mỗi cung $e \in E$ ngoài sức chứa (lưu lượng) tối đa $c(e)$ còn được gán một số không âm $d(e) \leq c(e)$ gọi là lưu lượng tối thiểu. Một *luồng tương thích* φ trên G được định nghĩa với tất cả các ràng buộc của luồng và thêm một điều kiện: Luồng trên mỗi cung $e \in E$ không được nhỏ hơn sức chứa tối thiểu của cung đó:

$$d(e) \leq \varphi(e) \leq c(e)$$

Bài toán đặt ra là kiểm chứng sự tồn tại của luồng tương thích trên mạng với ràng buộc luồng bị chặn hai phía.

Xây dựng một mạng G' từ mạng G theo quy tắc:

- ☀ Tập đỉnh V' có được từ tập V thêm vào đỉnh phát giả s^* và đỉnh thu giả t^* .
- ☀ Mỗi cung $e = (u, v)$ trên G sẽ tương ứng với ba cung trên G' : cung $e_1 = (u, v)$ có sức chứa $c(e) - d(e)$, cung $e_2 = (s^*, v)$ và cung $e_3 = (u, t^*)$ có sức chứa $d(e)$. Ngoài ra thêm vào cung (t, s) trên G' với sức chứa $+\infty$



Hình 1-10. Biến đổi mạng mới từ mạng với sức chứa các cung bị chặn hai phía

Gọi $D = \sum_{e \in E} d(e)$ là tổng sức chứa tối thiểu của các cung trên mạng G . Khi đó trên mạng G' , tổng sức chứa các cung đi ra khỏi s^* cũng như tổng sức chứa các cung đi vào t^* bằng D . Vì vậy với mọi luồng trên G' thì giá trị luồng đó không thể vượt quá D .

Từ đó suy ra rằng nếu tồn tại một luồng φ' trên G' có giá trị $|\varphi'| = D$ thì φ' bắt buộc là luồng cực đại trên G' .

Bổ đề 1-21

Điều kiện cần và đủ để tồn tại luồng tương thích φ trên mạng G là tồn tại luồng cực đại φ' trên G' với $|\varphi'| = D$.

Chứng minh

Giả sử luồng cực đại φ' trên G' có $|\varphi'| = D = \sum_{e \in E} d(e)$. Ta xây dựng luồng φ trên G bằng cách cộng thêm vào luồng φ' trên mỗi cung e một lượng $d(e)$:

$$\begin{aligned} \varphi: E &\rightarrow [0, +\infty) \\ e &\rightarrow \varphi(e) = \varphi'(e) + d(e) \end{aligned}$$

Khi đó có thể dễ dàng kiểm chứng được φ thỏa mãn tất cả các ràng buộc của luồng tương thích trên mạng G .

Ngược lại nếu φ là một luồng tương thích trên G . Ta xây dựng luồng φ' trên G' bằng cách trừ luồng φ trên mỗi cung e đi một lượng $d(e)$, đồng thời đặt luồng φ' trên các cung đi ra khỏi s' cũng như trên các cung đi vào t' đúng bằng sức chứa của cung đó. Khi đó cũng dễ dàng kiểm chứng được φ' là luồng cực đại và $|\varphi'| = D$.

1.6.4. Mạng với sức chứa âm

Cho mạng $G = (V, E, c, w, s, t)$ trong đó ta mở rộng khái niệm sức chứa bằng cách cho phép cả những sức chứa âm trên một số cung. Khái niệm luồng được định nghĩa như bình thường.

Để chỉ ra một luồng trên G , xét một cung $e \in E$ có sức chứa $c(e) < 0$. Theo tính phản đối xứng của luồng $f(e) = -f(-e)$ và ràng buộc sức chứa tối đa $f(e) \leq c(e)$, ta có:

$$f(-e) = -f(e) \geq -c(e) > 0 \quad (1.1)$$

Như vậy ràng buộc sức chứa tối đa $f(e) \leq c(e)$ tương đương với ràng buộc về sức chứa tối thiểu $-c(e)$ trên cung đối $-e$. Việc chỉ ra một luồng khởi tạo trên G có thể thực hiện bằng cách tìm luồng với ràng buộc luồng bị chặn hai phía:

$$-c(e) \leq f(-e) \leq c(-e)$$

Khi đã có luồng khởi tạo f :

- ☀ Để tìm luồng cực đại trên G , ta có thể tìm luồng cực đại d trên G_f (có sức chứa không âm) rồi trả về luồng $f + d$
- ☀ Để tìm luồng cực tiểu trên G , ta đảo vai trò đỉnh phát và đỉnh thu trên G_f (t phát, s thu) và tìm luồng cực đại d trên G_f (có sức chứa không âm) rồi trả về luồng $f + d$

Kỹ thuật này cũng có thể sử dụng để tìm luồng khả thi cực đại/cực tiểu trên mạng với luồng bị chặn hai phía (Mục 1.6.3).

1.6.5. Lát cắt hẹp nhất

Ta quan tâm tới đồ thị vô hướng liên thông $G = (V, E)$ với hàm trọng số (hay lưu lượng) $c: E \rightarrow [0, +\infty)$. Giả sử $|V| \geq 2$, người ta muốn bỏ đi một số cạnh để đồ thị mất tính liên thông và yêu cầu tìm phương án sao cho tổng trọng số các cạnh bị loại bỏ là nhỏ nhất.

Bài toán cũng có thể phát biểu dưới dạng: hãy phân hoạch tập đỉnh V thành hai tập khác rỗng rời nhau X và Y sao cho tổng lưu lượng các cạnh nối giữa X và Y là nhỏ nhất có thể. Cách phân hoạch này gọi là lát cắt tổng quát hẹp nhất của G , ký hiệu $MinCut(G)$.

$$c(X, Y) \rightarrow \min$$

$$X \neq \emptyset; Y \neq \emptyset; X \cap Y = \emptyset; X \cup Y = V;$$

Một cách tệ nhất có thể thực hiện là thử tất cả các cặp đỉnh s, t . Với mỗi lần thử ta cho s làm đỉnh phát và t làm đỉnh thu trên mạng G , sau đó tìm luồng cực đại và lát cắt $s - t$ hẹp nhất. Cuối cùng là chọn lát cắt $s - t$ có lưu lượng nhỏ nhất trong tất cả các lần thử. Phương pháp này cần $\binom{n}{2} = \frac{n \times (n-1)}{2}$ lần tìm luồng cực đại, có tốc độ chậm và không khả thi với dữ liệu lớn.

Bổ đề 1-22

Với s và t là hai đỉnh bất kỳ. Từ đồ thị G , ta xây dựng đồ thị G_{st} bằng cách chập hai đỉnh s và t thành một đỉnh duy nhất, ký hiệu st , các cạnh nối s với t bị hủy bỏ, các cạnh liên thuộc với chỉ s hoặc t được chỉnh lại đầu mút để trở thành cạnh liên thuộc với st . Khi đó $MinCut(G)$ có thể thu được bằng lấy lát cắt có lưu lượng nhỏ nhất trong hai lát cắt:

- ☀ Lát cắt $s - t$ hẹp nhất: Coi s là đỉnh phát và t là đỉnh thu, lát cắt $s - t$ hẹp nhất có thể xác định bằng việc giải quyết bài toán luồng cực đại trên mạng G .
- ☀ Lát cắt tổng quát hẹp nhất trên G_{st} : $MinCut(G_{st})$.

Chứng minh

Xét lát cắt tổng quát hẹp nhất trên G có thể đưa s và t vào hai thành phần liên thông khác nhau hoặc đưa chúng vào cùng một thành phần liên thông. Trong trường hợp thứ nhất, $MinCut(G)$ là lát cắt $s - t$ hẹp nhất. Trong trường hợp thứ hai, $MinCut(G)$ là $MinCut(G_{st})$.

Bổ đề 1-22 cho phép chúng ta xây dựng một thuật toán tốt hơn: Nếu đồ thị chỉ gồm 2 đỉnh thì chỉ việc cắt rời hai đỉnh vào hai tập. Nếu không, ta chọn hai đỉnh bất kỳ s, t làm đỉnh phát và đỉnh thu, tìm luồng cực đại và ghi nhận lát cắt $s - t$ hẹp nhất. Tiếp theo ta chập hai đỉnh s, t thành một đỉnh st và lặp lại với đồ thị G_{st} ... Cuối cùng là chỉ ra lát cắt $s - t$ hẹp nhất trong số tất cả các lát cắt được ghi nhận. Phương pháp này đòi hỏi phải thực hiện $|V| - 1$ lần tìm luồng cực đại, tuy đã có sự cải thiện về tốc độ nhưng chưa phải thật tốt.

Nhận xét rằng tại mỗi bước của cách giải trên, chúng ta có thể chọn hai đỉnh s, t bất kỳ miễn sao $s \neq t$. Vì vậy người ta muốn tìm một cách chọn cặp đỉnh s, t một cách hợp lý tại mỗi bước để có thể chỉ ra ngay lát cắt $s - t$ hẹp nhất mà không cần tìm luồng cực đại. Thuật toán dưới đây [5] là một trong những thuật toán hiệu quả dựa trên ý tưởng đó.

Với A là một tập con của tập đỉnh V và x là một đỉnh không thuộc A . Định nghĩa *lực hút* của A đối với x là tổng trọng số các cạnh nối x với các đỉnh thuộc A : $c(A, x)$

Bổ đề 1-23

Bắt đầu từ tập A chỉ gồm một đỉnh bất kỳ $a \in V$, ta cứ tìm một đỉnh bị A hút chặt nhất kết nạp thêm vào A cho tới khi $A = V$. Gọi s và t là hai đỉnh được kết nạp cuối cùng theo cách này. Khi đó lát cắt $(V - \{t\}, \{t\})$ là lát cắt $s - t$ hẹp nhất.

Chứng minh

Xét một lát cắt $s - t$ bất kỳ (X, Y) , ta sẽ chứng minh rằng lưu lượng của lát cắt $(V - \{t\}, \{t\})$ không lớn hơn lưu lượng của lát cắt (X, Y) , tức là:

$$c(V - \{t\}, t) \leq c(X, Y)$$

Một đỉnh v được gọi là *đỉnh hoạt tính* nếu v và đỉnh được đưa vào A liền trước v bị rơi vào hai phía của lát cắt (X, Y) . Xét thời điểm sau khi v được kết nạp vào A , gọi:

- ☀ A_v bằng $A - \{v\}$
- ☀ X_v là các đỉnh $\in X$ được kết nạp vào A
- ☀ Y_v là các đỉnh $\in Y$ được kết nạp vào A

Trước hết ta sử dụng phép quy nạp để chỉ ra rằng nếu u là đỉnh hoạt tính thì:

$$c(A_u, u) \leq c(X_u, Y_u) \quad (1.2)$$

Nếu u là đỉnh hoạt tính đầu tiên được kết nạp vào A , ta có u nằm ở một phía của lát cắt (X, Y) và A_u chứa các đỉnh nằm ở phía còn lại. Tức là cặp $(A_u, \{u\})$ cũng là cặp (X_u, Y_u) nên $c(A_u, u) = c(X_u, Y_u)$.

Giả thiết rằng bất đẳng thức (1.2) đúng với đỉnh hoạt tính u , ta sẽ chứng minh nó cũng đúng với những đỉnh hoạt tính v được kết nạp vào A sau u . Thật vậy:

$$c(A_v, v) = c(A_u, v) + c(A_v - A_u, v) \quad (1.3)$$

Do A_u phải hút u mạnh hơn hay bằng v , kết hợp với giả thiết quy nạp, ta có:

$$c(A_u, v) \leq c(A_u, u) \leq c(X_u, Y_u)$$

Hạng tử $c(A_v - A_u, \{v\})$ là tổng trọng số các cạnh nối giữa v và $A_v - A_u$. Do u và v là hai đỉnh hoạt tính liên tiếp, các cạnh này sẽ nối giữa hai tập X_v, Y_v . Mặt khác khi u được kết nạp vào A thì v chưa được kết nạp nên những cạnh này không nối giữa hai tập X_u, Y_u . Tổng hợp lại ta có: sức chứa của những cạnh nối giữa v và $A_v - A_u$ có mặt trong phép tính:

$$c(X_v, Y_v) = \sum_{\forall e \in \{X_v \rightarrow Y_v\}} c(e)$$

nhưng không có mặt trong phép tính:

$$c(X_u, Y_u) = \sum_{\forall e \in \{X_u, Y_u\}} c(e)$$

Vậy:

$$\begin{aligned} c(A_v, v) &= c(A_u, v) + c(A_v - A_u, v) \\ &\leq c(X_u, Y_u) + c(A_v - A_u, v) \\ &\leq c(X_v, Y_v) \end{aligned} \quad (1.4)$$

Vì (X, Y) là một lát cắt $s - t$ nên s và t nằm ở hai phía khác nhau của lát cắt (X, Y) . Ngoài ra s và t là hai đỉnh được kết nạp vào A sau cùng nên t là đỉnh hoạt tính. Bất đẳng thức (1.4) chứng minh ở trên cho ta kết quả:

$$c(V - \{t\}, t) = c(A_t, t) \leq c(X_t, Y_t) = c(X, Y)$$

Ta chứng minh được lát cắt $(V - \{t\}, \{t\})$ là lát cắt $s - t$ hẹp nhất.

Định lý 1-24

Lát cắt tổng quát trên đồ thị vô hướng liên thông với hàm trọng số không âm có thể tìm được bằng thuật toán trong thời gian $O(|V|^2 \log|V| + |V||E|)$.

Chứng minh

Bắt đầu từ tập A chỉ gồm một đỉnh bất kỳ, ta mở rộng A bằng cách lần lượt kết nạp vào A đỉnh bị hút chặt nhất cho tới khi $A = V$. Việc này được thực hiện với kỹ thuật gán nhãn lực hút: với $\forall v \notin A$, ta ký hiệu nhãn $d[v]$ là lực hút của A đối với đỉnh v . khi A được kết nạp thêm một đỉnh u thì các nhãn lực hút của những đỉnh v khác sẽ được cập nhật lại theo công thức:

$$d[v]_{\text{mới}} = d[v]_{\text{cũ}} + c(u, v), \forall (u, v) \in E$$

Bằng việc tổ chức các đỉnh ngoài A trong một hàng đợi ưu tiên dạng Fibonacci Heap, việc mở rộng tập A cho tới khi $A = V$ được thực hiện trong thời gian $O(|V| \log |V| + |E|)$. Trong quá trình đó, s và t là hai đỉnh cuối cùng được kết nạp vào A cũng được xác định và $MinCut(G)$ được cập nhật theo lát cắt $s - t$ hẹp nhất. Sau đó hai đỉnh s, t được chập vào và thuật toán lặp lại với đồ thị G_{st} . Tổng cộng ta có $|V| - 1$ lần lặp, suy ra lát cắt tổng quát hẹp nhất có thể tìm được trong thời gian $O(|V|^2 \log |V| + |V||E|)$.

Mặc dù tính đúng đắn của thuật toán được chứng minh dựa vào lý thuyết về luồng cực đại và lát cắt hẹp nhất, việc cài đặt thuật toán lại khá đơn giản và không động chạm gì đến luồng cực đại.

Bài tập 1-1

Chỉ ra rằng nếu giữa 2 đỉnh (u, v) có nhiều cung song song, ta có thể thay thế các cung song song này bằng một cung (u, v) duy nhất có sức chứa bằng tổng sức chứa các cung song song ban đầu mà không làm ảnh hưởng tới giá trị luồng cực đại. Từ đó tìm cách cài đặt thuật toán giải quyết bài toán luồng cực đại trên mạng vô hướng mà không cần thêm vào các cung đối.

Bài tập 1-2

Cho f_1 và f_2 là hai luồng trên mạng $G = (V, E, c, s, t)$ và α là một số thực nằm trong đoạn $[0, 1]$. Xét ánh xạ:

$$\begin{aligned} f_\alpha: E &\rightarrow \mathbb{R} \\ e &\mapsto f_\alpha(e) = \alpha f_1(e) + (1 - \alpha) f_2(e) \end{aligned}$$

Chứng minh rằng f_α cũng là một luồng trên mạng G với giá trị luồng:

$$|f_\alpha| = \alpha |f_1| + (1 - \alpha) |f_2|$$

Bài tập 1-3

Cho f là luồng cực đại trên mạng $G = (V, E, c, s, t)$, gọi Y là tập các đỉnh đến được t bằng một đường dư trên G_f và $X = V - Y$. Chứng minh rằng (X, Y) là lát cắt $s - t$ hẹp nhất của mạng G .

Bài tập 1-4

Viết chương trình nhận vào một đồ thị có hướng $G = (V, E)$ với hai đỉnh phân biệt s và t và tìm một tập gồm nhiều đường đi nhất từ s tới t sao cho các đường đi trong tập này đôi một không có cạnh chung.

Gợi ý

Coi s là đỉnh phát và t là đỉnh thu, các cung đều có sức chứa 1 và tìm luồng cực đại trên mạng, theo Định lý 1-20 (Định lý về tính nguyên), luồng trên các cung chỉ có thể là 0 hoặc 1. Loại bỏ các cung có luồng 0 và chỉ giữ lại các cung có luồng 1. Tiếp theo ta tìm một đường đi từ s tới t , chọn đường đi này vào tập hợp, loại bỏ tất cả các cung dọc trên đường đi này khỏi đồ thị và lặp lại..., thuật toán sẽ kết thúc khi đồ thị không còn cạnh nào (không còn đường đi từ s tới t).

Bài tập 1-5

Tương tự như Bài tập 1-4 nhưng yêu cầu thực hiện trên đồ thị vô hướng.

Bài tập 1-6 (Hệ đại diện phân biệt)

Một lớp học có n bạn nam và n bạn nữ. Nhân ngày 8/3, lớp có mua m món quà để các bạn nam tặng các bạn nữ. Mỗi món quà có thể thuộc sở thích của một số bạn trong lớp.

Hãy lập chương trình tìm cách phân công tặng quà thỏa mãn:

- ☀ Mỗi bạn nam phải tặng quà cho đúng một bạn nữ và mỗi bạn nữ phải nhận quà của đúng một bạn nam. Món quà được tặng phải thuộc sở thích của cả hai người.
- ☀ Món quà nào đã được một bạn nam chọn để tặng thì bạn nam khác không được chọn nữa.

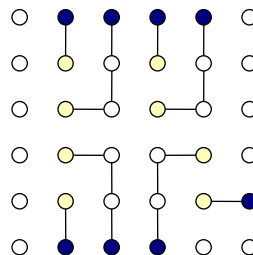
Gợi ý: Xây dựng một mạng trong đó tập đỉnh V gồm 3 lớp đỉnh S, X và T :

- ☀ Lớp đỉnh phát $S = \{s_1, s_2, \dots, s_n\}$, mỗi đỉnh tương ứng với một bạn nam.
- ☀ Lớp đỉnh $X = (x_1, x_2, \dots, x_n)$ mỗi đỉnh tương ứng với một món quà.
- ☀ Lớp đỉnh thu $T = \{t_1, t_2, \dots, t_n\}$ mỗi đỉnh tương ứng với một bạn nữ.

Nếu bạn nam i thích món quà k , ta cho cung nối từ s_i tới x_k , nếu bạn nữ j thích món quà k , ta cho cung nối từ x_k tới t_j . Sức chứa của các cung đặt bằng 1 và sức chứa của các đỉnh $v_1, v_2, \dots, v_n$ cũng đặt bằng 1. Tìm luồng nguyên cực đại trên mạng G có n đỉnh phát, n đỉnh thu, đồng thời có cả ràng buộc sức chứa trên các đỉnh, những cung có luồng 1 sẽ nối từ một người tặng tới một món quà và từ một món quà với người nhận tương ứng.

Bài tập 1-7

Cho mạng điện gồm $m \times n$ điểm nằm trên một lưới m hàng, n cột. Một số điểm nằm trên biên của lưới là nguồn điện, một số điểm trên lưới là các thiết bị sử dụng điện. Người ta chỉ cho phép nối dây điện giữa hai điểm nằm cùng hàng hoặc cùng cột. Hãy tìm cách đặt các dây điện nối các thiết bị sử dụng điện với nguồn điện sao cho hai đường dây bất kỳ nối hai thiết bị sử dụng điện với nguồn điện tương ứng của chúng không được có điểm chung.



Bài tập 1-8 (Kỹ thuật giãn sức chứa)

Cho mạng $G = (V, E, c, w, s, t)$ với sức chứa nguyên: $c: E \rightarrow \mathbb{N}$. Gọi $C = \max_{e \in E} \{c(e)\}$.

- Chứng minh rằng lát cắt $s - t$ hẹp nhất của G có lưu lượng không vượt quá $C \times |E|$
- Với một số nguyên k , tìm thuật toán xác định đường tăng luồng có dư lượng $\geq k$ trong thời gian $O(|E|)$.
- Chứng minh rằng thuật toán sau đây tìm được luồng cực đại trên mạng G :

```
void MaxFlowByScaling()
{
    f = «Luồng 0»;
    for (k = C; k ≥ 1; k = ⌊k / 2⌋)
        while («Tìm được đường tăng luồng P có dư lượng ≥ k»)
            «Tăng luồng dọc đường P»;
}
```

d) Khi bước vào mỗi lượt lặp của vòng lặp for..., gọi (X, Y) là lát cắt $s - t$ hẹp nhất trên mạng dư lượng G_f . Chứng minh rằng $c_f(X, Y) \leq 2k|E|$.

e) Chứng minh rằng trong mỗi lượt lặp của vòng lặp for, vòng lặp while bên trong thực hiện $O(|E|)$ lần

f) Chứng minh rằng thuật toán trên (*maximum flow by scaling*) có thể cài đặt để tìm luồng cực đại trên G trong thời gian $O(|E|^2 \log C)$.

Bài tập 1-9 (Chọn dự án)

Cho n dự án đánh số từ 1 tới n và m máy đánh số từ 1 tới m . Dự án thứ i khi được chọn thực hiện sẽ thu được khoản tiền là r_i , máy thứ j khi mua sẽ phải chi ra khoản tiền là c_j . Một dự án muốn thực hiện có thể cần mua một vài máy và khi một máy đã mua có thể dùng trong nhiều dự án khác nhau. Hãy chọn ra các máy cần mua và các dự án cần thực hiện để số tiền lợi nhuận (sau khi cân đối thu chi) là lớn nhất.

Gợi ý:

Trong một phương án bất kỳ, gọi P là tập các dự án không được làm và Q là tập các máy được mua, khi đó ta cần tìm phương án để cực đại hóa lợi nhuận:

$$\sum_{i=1}^n r_i - \sum_{i \in P} r_i - \sum_{j \in Q} c_j \rightarrow \max$$

Vì giá trị $\sum_{i=1}^n r_i$ không phụ thuộc phương án, ta có thể đổi hàm mục tiêu thành:

$$\sum_{i \in P} p_i + \sum_{j \in Q} q_j \rightarrow \min$$

Dựng mạng trong đó mỗi dự án i ứng với một đỉnh p_i , mỗi máy j ứng với một đỉnh q_j , thêm một đỉnh phát s và đỉnh thu t .

- ☀ Với mọi dự án i , thêm cung (s, p_i) với sức chứa r_i ($\forall i: 1 \leq i \leq n$)
- ☀ Với mọi máy j , thêm cung (q_j, t) với sức chứa c_j ($\forall j: 1 \leq j \leq m$)
- ☀ Nếu dự án i cần máy j , thêm cung (p_i, q_j) với sức chứa $+\infty$ ($\forall i, j: 1 \leq i \leq n; 1 \leq j \leq m$)

Tìm luồng cực đại trên mạng và xác định một lát cắt $s - t$ hẹp nhất (X, Y)

a) Chỉ ra rằng những cung $\in \{X \rightarrow Y\}$ chỉ thuộc một trong hai dạng: cung (s, p_i) hoặc cung (q_j, t)

b) Với một phương án bất kỳ, gọi P là tập các dự án không được làm và Q là tập các máy được mua. Chứng minh điều kiện cần và đủ để phương án P hợp lệ là: với mọi cung (p_i, q_j) thì hoặc $i \in P$ hoặc $j \in Q$.

c) Xét những cung $\in \{X \rightarrow Y\}$, đặt:

$$P = \{i: (s, p_i) \in (X, Y)\}$$

$$Q = \{j: (q_j, t) \in (X, Y)\}$$

Chỉ ra rằng phương án tối ưu sẽ là phương án mà những dự án $\in P$ không được làm và những máy $\in Q$ được mua.

Bài tập 1-10 (Chọn dự án 2)

Có n dự án đánh số từ 1 tới n , dự án thứ i khi làm xong sẽ thu được lợi nhuận là r_i (r_i có thể âm nếu số tiền chi ra để đầu tư ban đầu lớn hơn số tiền thu lại sau khi hoàn thành dự án). Có m mối quan hệ, mỗi quan hệ có dạng (i, j) cho biết muốn chọn dự án i thì bắt buộc phải chọn cả dự án j . Hãy chọn một số dự án để làm để tổng lợi nhuận thu được là lớn nhất.

Gợi ý:

Dựng mạng với n đỉnh tương ứng với n dự án, thêm một đỉnh phát $s = 0$ và một đỉnh thu $t = n + 1$, coi như $r_0 = r_{n+1} = 0$. Các cung của mạng được xây dựng như sau:

- ☀ Với mỗi quan hệ (i, j) (nếu muốn chọn dự án i thì phải chọn dự án j), ta thêm cung (i, j) với sức chứa $+\infty$.
- ☀ Với mỗi dự án i có lợi nhuận $r_i \geq 0$, thêm cung (s, i) với sức chứa r_i ($\forall i: 1 \leq i \leq n$)
- ☀ Với mỗi dự án j có lợi nhuận $r_j < 0$, thêm cung (j, t) với sức chứa $-r_j$ ($\forall j: 1 \leq j \leq n$)

Với một phương án hợp lệ để chọn dự án, gọi Y là tập gồm đỉnh t và các dự án được chọn, tập X gồm dự án không được chọn và đỉnh s . Ta xây dựng được (X, Y) là lát cắt $s - t$ ứng với một phương án chọn dự án.

a) Chứng minh rằng $c(X, Y) < +\infty$, tức là không tồn tại cung với sức chứa $+\infty$ nối từ X sang Y . Nói cách khác, những cung nối từ X sang Y hoặc là cung dạng (s, i) , hoặc là cung dạng (j, t) .

b) Xét biểu thức của $c(X, Y)$

$$\begin{aligned} c(X, Y) &= \sum_{\forall i \in Y: r_i \geq 0} c(s, i) + \sum_{\forall j \in X: r_j < 0} c(j, t) \\ &= \sum_{\forall i \in X: r_i \geq 0} r_i + \sum_{\forall j \in Y: r_j < 0} (-r_j) \\ &= \underbrace{\sum_{\forall i: r_i \geq 0} r_i}_{\text{Hằng số } C} - \sum_{\forall i \in Y: r_i \geq 0} r_i - \sum_{\forall j \in Y: r_j < 0} r_j \\ &= C - \sum_{\forall i \in Y} r_i \end{aligned}$$

Chỉ ra rằng nếu lưu lượng của lát cắt (X, Y) càng nhỏ thì lợi nhuận của phương án tương ứng càng lớn.

c) Xây dựng thuật toán giải quyết bài toán này thông qua mô hình luồng cực đại và lát cắt hợp nhất.